arXiv:2604.17338v5 [cs.SE] 5 Oct 2026

Precise Debugging Benchmark: Is Your Model Debugging or Regenerating?

Miaosen Chai[*♠] **Wang Bill Zhu**[*♠] **Shangshang Wang**[♠] **Yejia Liu**[§]
Song Bian[♡] **Honghua Dong**[◇] **Willie Neiswanger**[♠] **Robin Jia**[♠]
[♠]University of Southern California [§]Microsoft
[♡]University of Wisconsin–Madison [◇]University of Toronto

Dataset **Webpage** **Code** **Leaderboard**

Abstract

Unlike code completion, debugging requires localizing faults and applying targeted edits. We observe that frontier LLMs often hack unit tests by *regenerating* correct but over-edited solutions during debugging. To evaluate how far LLMs are from precise debugging, we introduce the **PRECISE DEBUGGING BENCHMARKING (PDB)**, a general, *dataset-agnostic* framework that converts *any* coding dataset into a debugging benchmark with precision-aware evaluation. PDB generates buggy programs by synthesizing verified atomic bugs and composing them into multi-bug programs. We define two novel metrics, edit-level *precision* and bug-level *recall*, which measure how many necessary edits are made and how many bugs are resolved. We release two evaluation benchmarks: **PDB-SINGLE** on single-line bugs, and **PDB-WILD** on multi-line and repository-level bugs. Experiments show that frontier models, such as GPT-5.1-Codex and DeepSeek-V3.2-Thinking, achieve unit-test pass rates above 76% but exhibit precision below 45%, even when explicitly instructed to perform minimal debugging on single-line bugs. Iterative and agentic debugging strategies do not substantially improve precision or recall, highlighting the need to rethink post-training pipelines for coding models.

1 Introduction

Large Language Models (LLMs) have reshaped the programming landscape through their remarkable capabilities in code generation [Chen et al., 2021, Li et al., 2023]. From synthesizing complex algorithms from natural language prompts to translating entire codebases, modern LLMs excel at producing code from scratch. However, real-world software development is dominated not by generation but by debugging and maintenance [Glass, 2002]. When applied to debugging tasks, we observe that frontier LLMs often default to regeneration, *i.e.*, rewriting large portions, or even the entirety, of a program when presented with buggy code (Figure 1). While often effective at passing tests, this brute-force strategy is poorly suited for realistic codebases, where large-scale rewrites are costly, risky, and difficult to review [Sobania et al., 2023]. In contrast, targeted debugging requires precise fault localization and minimal, intent-preserving edits. This raises a fundamental question: *How far are LLMs from precise debugging, rather than merely reverting to their strength in code regeneration?*

Existing debugging benchmarks focus on unit-test only evaluation and fail to evaluate these capabilities. Under such evaluation, models are rewarded equally for regenerating a full solution, hard-coding

[*] Equal contribution.
[*] Corresponding authors: miaosenc@uchicago.edu, wangzhu@berkeley.edu.
[†] Work done before joining Microsoft.

outputs, or performing a minimal targeted fix. Moreover, unit-test evaluation obscures incremental progress: a model that correctly repairs only one defect in a multi-bug program receives the same score as a model that fixes none. This misalignment with real-world debugging practice limits our ability to understand how LLMs reason about bugs and code edits. Also, existing debugging benchmarks are predominantly derived from publicly accessible debugging questions and answers collected from platforms such as Stack Overflow and GitHub issues [Jimenez et al., 2023, Yasunaga and Liang, 2021, Phan et al., 2021]. They are vulnerable to contamination and require extensive manual effort to construct.

To address these gaps, we introduce the PRECISE DEBUGGING BENCHMARKING (PDB) framework, a general pipeline that rigorously evaluates LLM debugging behavior independently of code generation. PDB provides a dataset-agnostic plug-and-play framework that converts existing coding datasets into debugging benchmarks through two steps: (1) synthesizing verified atomic bugs to produce ground-truth edit scripts, and (2) composing these bugs into multi-bug programs while preserving bug independence (*i.e.*, avoiding compounding interactions). Beyond binary test outcomes, PDB evaluates model patches using novel edit-level *precision* and bug-level *recall*, explicitly rewarding targeted fixes and penalizing unnecessary modifications.

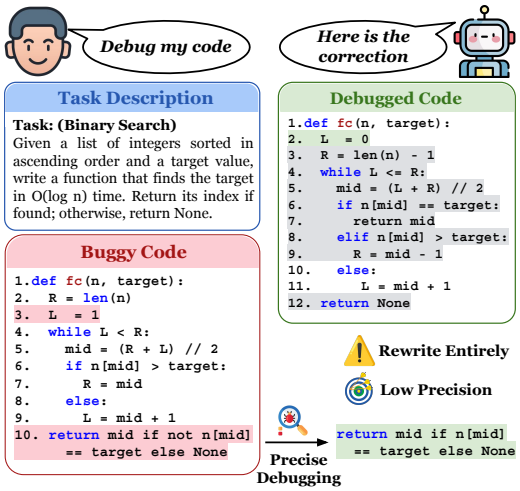


Figure 1: Real example from GPT-5.2 debugging a binary search program, where the model rewrites the entire solution. Green lines mark precise edits; gray lines highlight over-edits.

We constructed a 5,751-example PDB-SINGLE and a 484-example PDB-WILD evaluation benchmarks with the PDB framework, from BigCodeBench [Zhuo et al., 2024], LiveCodeBench [Jain et al., 2024], and SWE-smith [Yang et al., 2025a]. Experiments on PDB-SINGLE reveal behaviors that unit tests fail to capture. First, frontier models exhibit strikingly **different rankings under edit-level evaluation**. Models such as GPT-5.1-Codex OpenAI [2025a] and DeepSeek-V3.2-Thinking DeepSeek-AI [2025] achieve high unit-test pass rates ($>76\%$) but low edit precision ($\leq 45\%$), while Qwen3-Coder-480B Qwen [2025] attains comparatively lower unit-test pass rates (70%) yet substantially higher precision (66%). This ranking inversion **persists** on the multi-line and repository-level PDB-WILD benchmark, including when newer models such as GPT-5.5 and Claude-Opus-4.7 are evaluated on bugs generated by the models themselves. Additionally, we show that though iterative and agentic debugging strategies can improve unit-test performance, they do not meaningfully improve precision or recall. Our findings demonstrate the necessity of PDB for revealing true debugging capabilities beyond surface-level correctness, and highlight a fundamental limitation in current post-training pipelines for coding LLMs.

2 Precise Debugging Setup

We begin by formally defining the components of the automated debugging task. An automated debugging system $\mathcal{M}$, takes the initial buggy program C_{b} and a natural language task description x as input, and returns the predicted program revision $\hat{C} = \mathcal{M}(C_{\text{b}}, x)$. The conventional debugging pipeline evaluates the system's final output purely on its functional correctness, using a binary evaluation function $F_{\mathcal{U}}(C) \to \{0, 1\}$, where $\mathcal{U} = \{u_1, u_2, \ldots, u_n\}$ is a suite of designed unit tests. The evaluation function $F_{\mathcal{U}}$ returns 1 if the program C passes all tests in $\mathcal{U}$, and 0 otherwise.

While straightforward, this method cannot penalize unnecessary edits or wholesale rewrites when the final program merely passes the test, nor can it distinguish between partially correct solutions and entirely incorrect ones. The precise debugging setup shifts the evaluation from program-level to the specific set of edits proposed by the model.

Minimal corrections. We denote a line-edit on line l as e_l, and a set of line-edits as E. For a buggy program C_{b}, we denote the set of *minimal corrections* by $\mathcal{E}_{C_{\text{b}}} =$

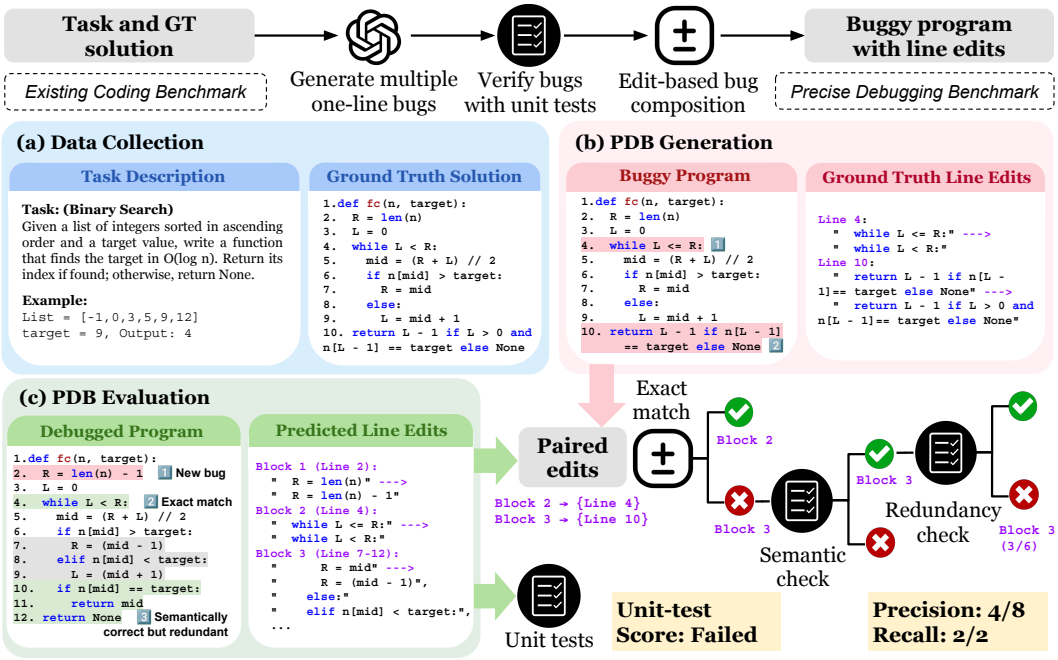


Figure 2: PDB pipeline. Generation: LLMs first synthesize and verify single-line bugs from existing coding datasets, which are then composed into multi-bug programs. Evaluation: Automated debugging systems are evaluated on these programs using both unit-test accuracy and edit-level precision and bug-level recall.

$\arg\min_E |E|$ s.t. $F_{\mathcal{U}}(\text{apply}(E, C_\text{b})) = 1$, where the apply function applies line-edits on C_b. Similarly, we can apply reverse edits $\bar{E}$ on ground-truth program C_gt to derive buggy program C_b.

Atomicity. We define the bug in a buggy program C_b as *atomic* when a minimal correction consists of edits on a contiguous sequence of lines. Formally, $\exists E \in \mathcal{E}_{C_\text{b}}$ such that $E = \{e_i, e_{i+1}, \ldots, e_{i+n}\}$.

Independence. Intuitively, independence means that fixing one bug neither introduces nor removes edits required to fix the other. For two edit sets $E_1 \in \mathcal{E}_{C_{\text{b}1}}$ and $E_2 \in \mathcal{E}_{C_{\text{b}2}}$ corresponding to the same ground-truth program C_gt, we can construct a composed buggy program $C_{\text{b}3} = \text{apply}(\bar{E}_1 \cup \bar{E}_2, C_\text{gt})$. If the set of minimal corrections is the pairwise union of corrections from $\mathcal{E}_{C_{\text{b}1}}$ and $\mathcal{E}_{C_{\text{b}2}}$, we consider bugs in $C_{\text{b}1}$ and $C_{\text{b}2}$ to be *independent*.

Semantic correctness. Consider a buggy program C_b containing k atomic and independent bugs, and a revision $\hat{C} = \text{apply}(\hat{E}, C_\text{b})$, where $\hat{E}$ is the predicted edits. We define *bug-level semantic correctness* as follows.

Let $E_\text{gt} \in \mathcal{E}_{C_\text{b}}$ be the set of ground-truth edits, which can be decomposed as $E_\text{gt} = E_1 \cup E_2 \cup \cdots \cup E_k$, where $E_1, \ldots, E_k$ are contiguous and non-overlapping. We employ a function, denoted as map, which pairs each E_i with the closest edits in $\hat{E}$. For each bug i, we construct a pseudo-revision $\hat{C}_i = \text{apply}((E_\text{gt} \setminus E_i) \cup \text{map}(E_i), C_\text{b})$, which replaces the ground-truth edits E_i with the predicted edits $\text{map}(E_i)$. We define a candidate $\hat{C}_i$ as *semantically correct* for bug i if $F_{\mathcal{U}}(\hat{C}_i) = 1$. Figure 2 (Block 3) is such an example. Based on this, we define precision and recall as:

$$\text{precision} = \frac{1}{|\hat{E}|} \sum_{i=1}^{k} F_{\mathcal{U}}(\hat{C}_i) \cdot |E_i|, \tag{1}$$

$$\text{recall} = \frac{1}{k} \sum_{i=1}^{k} F_{\mathcal{U}}(\hat{C}_i). \tag{2}$$

We note that precision functions as an edit-level metric by averaging over the edits $|\hat{E}|$, while recall is a bug-level metric averaged over the k bugs.

ϵ-relaxed essential edits. Since our objective is to discourage solution *regeneration* rather than to enforce *strictly minimal* edits, we relax the precision metric in Eq. (1) by introducing a tolerance parameter ϵ, which allows up to $|E_i| + \epsilon$ edited lines for each bug i.

Moreover, even when a candidate revision $\hat{C}_i$ is semantically correct for bug i, the predicted edits $\mathrm{map}(E_i)$ may still contain regeneration, as illustrated in Figure 2 (Block 3). To remove such redundancy, we introduce a unit-test–based function $\mathrm{essential}_{\mathcal{U}}$, which searches over subsets of $\mathrm{map}(E_i)$ to recover the minimal essential edits required to resolve bug i while preserving semantic correctness. Formally, we define the *ϵ-relaxed essential edit size* for bug i as

$$(|\hat{E}_i|)_\epsilon = \min(|\,\mathrm{essential}_{\mathcal{U}}(\mathrm{map}(E_i))|, |E_i| + \epsilon).$$

Accordingly, the ϵ-relaxed precision is defined as

$$\text{precision}_\epsilon = \frac{1}{|\hat{E}|} \sum_{i=1}^{k} F_{\mathcal{U}}(\hat{C}_i) \cdot (|\hat{E}_i|)_\epsilon. \tag{3}$$

We provide full details of the map and $\mathrm{essential}_{\mathcal{U}}$ procedures in Appendix C.

3 Generation and Evaluation Pipeline

As illustrated in Figure 2, PDB consists of two stages: *generation* and *evaluation*. During the PDB generation stage, we first use LLMs to synthesize atomic bugs from existing coding datasets. After verifying buggy programs with unit tests, we record the corresponding edit sets and compose them to construct multi-bug programs. During the PDB evaluation stage, we prompt an automated debugging system $\mathcal{M}$ to revise the buggy programs, and evaluate its performance using both traditional unit-test accuracy and our proposed edit-level precision and bug-level recall metrics. All prompt templates are provided in Appendix E.

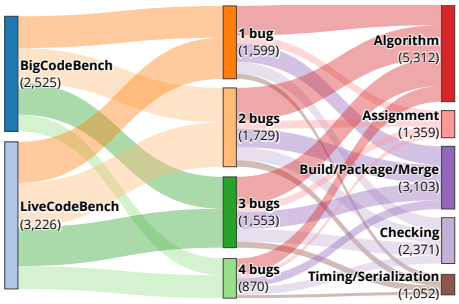
3.1 PDB generation

Starting from an existing coding benchmark, for each task description x and ground-truth program C_{gt}, we generate buggy programs across five Orthogonal Defect Classification (ODC; Chillarege et al., 1992) categories: *Assignment*, *Checking*, *Algorithm*, *Build/Package/Merge*, and *Timing/Serialization*. Each category further contains several subcategories, listed in Table 4.

Atomic bug generation. Single-line bug can ensure atomicity, as the minimal correction satisfies $|E| = 1$. We consider three types of line-level operations: *insertion*, *deletion*, and *substitution*. We first apply a rule-based filter to identify lines that are not safely deletable (*e.g.*, causing indentation errors) or not editable (*e.g.*, function headers). To promote diversity, we then randomly select (i) one operation type, (ii) one bug category, and (iii) a subset of editable lines compatible with the chosen operation. An LLM from a generator pool is prompted to modify one of the selected lines to produce a single-line buggy program. We repeat this process m_1 times per (x, C_{gt}) pair and retain only programs that fail unit tests, ensuring the validity of injected bugs.

Multiline bug generation. To extend the pipeline to multi-line bugs, we apply the same editing procedure to contiguous blocks of code. Specifically, we randomly select (i) a block size $B \in [2, B_{\max}]$, (ii) one primary and two auxiliary bug categories, and (iii) a valid range from which to sample a contiguous block of lines. Because multi-line edits — even within a contiguous block — do not inherently guarantee atomicity, we explicitly filter out violations. We implement an *atomicity filter* by enumerating all partial fixes that revert a strict subset of the modified lines back to C_{gt}, and retain a bug instance only if *all* such partial fixes still fail unit tests. This procedure removes non-atomic cases where fixing a subset of edits is sufficient to pass the tests, which would otherwise inflate both precision and recall.

Bug composition. To create more challenging debugging scenarios, we compose multiple atomic bugs into a single program. For each (x, C_{gt}) pair and a target bug count k, we randomly sample k distinct block edits from the generated bugs. To encourage independence between bugs, we enforce a *stride* constraint, requiring any two selected edits to be at least s lines apart. For each bug count $k \in \{2, \ldots, k_{\max}\}$, we repeat this process m_2 times per (x, C_{gt}) pair and record all composed multi-bug programs that satisfy the constraint.



(a)

Model	Precision	Recall	Unit (%)
Claude-Sonnet-4.5	**71.8±0.9**	81.4±0.8	75.7±1.1
Gemini-2.5-Pro	71.4±0.9	**83.5±0.7**	78.1±1.0
Qwen3-Coder-480B	65.8±0.9	77.2±0.9	70.3±1.2
Kimi-K2-Instruct	56.6±1.0	72.7±0.9	64.8±1.2
Grok-Code-Fast	54.6±1.0	66.5±1.0	58.3±1.3
Kimi-K2-Thinking	51.7±0.9	75.6±0.9	74.0±1.1
DeepSeek-V3.2	48.4±1.0	70.0±1.0	71.4±1.2
DeepSeek-V3.2-Thinking	45.0±0.9	71.2±1.0	**79.0±1.0**
GPT-5.1-Codex	39.7±0.8	71.7±0.9	76.1±1.1

(b)

Figure 3: (a) Data distribution of PDB-SINGLE. (b). Precision, recall, and unit score on the PDB-SINGLE set. Blue indicates better performance, while red indicates worse.

Subsampling. To avoid over-representation of (x, C_{gt}) pairs with many generations, we subsample the data by randomly selecting at most m_3 buggy programs per bug count per (x, C_{gt}) pair.

3.2 PDB evaluation

During evaluation, debugging systems, either single-pass LLMs or LLM-based agents, are instructed to debug a buggy program C_{b}, given the task description x and, optionally, access to unit tests $\mathcal{U}$ and unit-test error feedback.

We use the precision, recall equations as in Eq. (2, 3), and report the unit-test score at Pass@1 [Kulal et al., 2019]. All metrics are reported as averages over all examples in an evaluation set.

3.3 PDB test-suite

PDB separates two properties that are sometimes conflated. First, *instance detectability* asks whether the inherited tests expose the specific injected bug. PDB enforces this property per instance: a candidate is retained only if the complete injected program fails, and a multi-line candidate is atomic only if every strict partial repair also fails. A suite that cannot detect a proposed bug therefore cannot admit that bug to the benchmark. Second, *scoring adequacy* asks whether every test-passing model patch is functionally correct. When instantiating PDB on a new source dataset, test adequacy should be treated as an upstream requirement.

Although no finite execution-based benchmark can guarantee this property in general, we empirically estimate the false-positive rate of our released dataset to be 1.85% due to the upstream test-suite. Here, a false positive refers to a model-generated patch that passes the inherited tests despite being functionally incorrect. More details in B.5.

We also add an optional test-adequacy gate to the released pipeline's preprocessing stage. *Coverage gate*: before bug injection, run the suite on the ground-truth solution under a line tracer and require every editable line to be executed by at least one test; lines never executed cannot host a detectable bug and are excluded from injection. *Test augmentation*: for tasks that fail the coverage gate, generate additional tests with an LLM and keep only those that pass on the ground-truth solution in repeated runs, optionally as property-based tests; the augmented suite then feeds the standard PDB validation and scoring path.

4 Evaluation Sets

Using the PDB generation pipeline, we release two evaluation sets: PDB-SINGLE targets single-line bugs, while PDB-WILD extends the pipeline to contiguous multi-line bug blocks. We source tasks from three existing coding benchmarks: BigCodeBench [Zhuo et al., 2024] for API-call-heavy tasks, LiveCodeBench [Jain et al., 2024] for competitive programming, and SWE-smith [Yang et al., 2025a] for repository-level software engineering. Our bug-generation pool consists of four frontier LLMs: GPT-5.1-Codex [OpenAI, 2025a], Claude-Sonnet-4.5 [Anthropic, 2025], Gemini-2.5-Pro [Comanici et al., 2025], and Claude-Opus-4.7 for SWE-smith-specific generation.

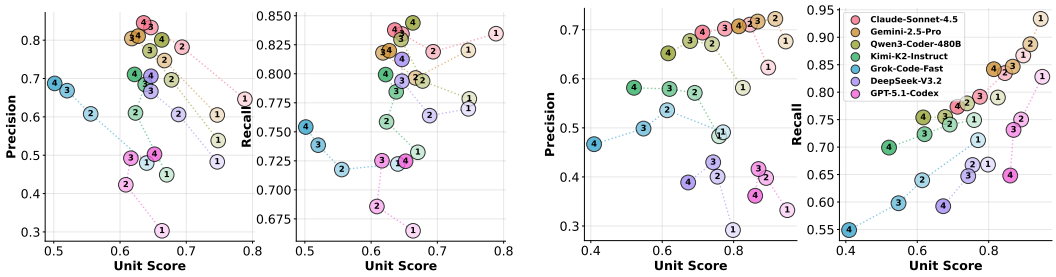


Figure 4: Correlation between precision, recall, and unit-test score across bug counts. Results are shown on subsets of PDB-SINGLE from BigCodeBench (left) and LiveCodeBench (right), with bug counts indicated by numbers. As the number of bugs increases, precision generally exhibits a negative correlation with unit-test score, while recall displays dataset-dependent behavior.

PDB-SINGLE. For each ground-truth task we generate $m_1 = 20$ single-line bugs, compose up to $m_2 = 100$ multi-bug variants with at most $k_{\max} = 4$ independent bugs per program, and subsample $m_3 = 5$ buggy programs per bug count per task. A stride of $s = 3$ lines is enforced between composed blocks. This yields the initial PDB-SINGLE-FULL set of 7,589 examples (see Appendix B.1 for details). We evaluate PDB-SINGLE-FULL on 9 models, including thinking models: GPT-5.1-Codex, Claude-Sonnet-4.5, Gemini-2.5-Pro, Grok-Code-Fast [xAI, 2025], DeepSeek-V3.2-Thinking, and Kimi-K2-Thinking [Kimi, 2025]; and non-thinking models: Qwen3-Coder-480B, DeepSeek-V3.2, and Kimi-K2-Instruct [Kimi, 2025]. All models are *prompted to produce minimal code edits*. We use a maximum output length of 32,000 tokens for thinking models and 8,000 tokens for non-thinking models, with a temperature of 1.0 throughout. We then apply model-based filtering to identify easy examples. We use a tolerance $\epsilon = 2$ for precision evaluation with Eq. (3) An example is labeled easy if it achieves perfect precision, recall, and unit-test score for at least 7 out of the 9 evaluated models. Applying this criterion removes 1,838 examples, resulting in the final PDB-SINGLE benchmark of 5,751 challenging examples.

PDB-WILD. For multi-line bugs, we select programs from BigCodeBench and LiveCodeBench with more than 35 lines and assign each generator a disjoint subset. We set the maximum block size to $B_{\max} = 4$, use stride $s = 5$ with the same m_1, m_2, m_3, and compose up to $k_{\max} = 3$ blocks per program, yielding 256 examples. To cover realistic software-engineering scenarios, we further sample 10 repositories from 128 public SWE-smith repositories [Yang et al., 2024, 2025a]; for each repository, we extract up to ten non-test source files and set $B_{\max} = 30$, $k_{\max} = 3$. Validated bugs survive in 32 files (110–1,222 lines, median 378) from 6 of the repositories, yielding 228 examples. Together, PDB-WILD contains 484 examples. Because multi-line blocks cannot strictly guarantee atomicity (for the up-to-30-line SWE-smith blocks we check single-line partial repairs only), we use a tolerance of $\epsilon = 1$.

5 Experiment Results

By evaluating and analyzing both LLMs and LLM-based agents on PDB-SINGLE and PDB-WILD, we show that current systems remain far from achieving precise, edit-aware debugging.

5.1 PDB-SINGLE overview

Divergence in model debugging behaviors. Even among top-performing models such as Claude-Sonnet-4.5 and Gemini-2.5-Pro, performance can only be characterized as relatively precise and faithful: no frontier model exceeds 72% precision, even when explicitly instructed to perform minimal debugging. Table 3b shows that unlike unit-test pass rates, edit-level precision and bug-level recall reveals four different types of model debugging behaviors:

- *Pass with precision*: Claude-Sonnet-4.5 and Gemini-2.5-Pro debug correctly (>75%) with the highest precision (>71%) and recall (>81%).
- *Weak but precise*: Qwen3-Coder-480B, though only achieves 70% unit score, has moderately high precision (66%) and recall (77%).

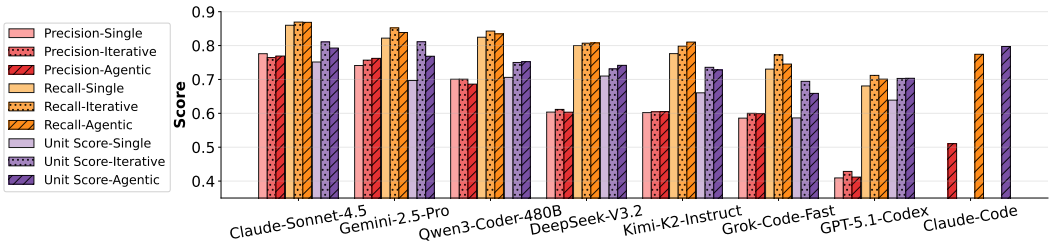


Figure 6: Both iterative and agentic setups on PDB-SINGLE improve unit-test pass rates and recall over single-shot debugging, indicating higher functional success. However, edit-level precision does not improve and sometimes degrades. Notably, even Claude-Code with access to unit-test and execution feedback exhibits only 50% precision.

- *Weak, imprecise, but identifying*: Kimi-K2-Instruct, Kimi-K2-Thinking, and Grok-Code-Fast reliably identify buggy regions but struggle to produce correct and precise fixes, with precision below 57%.
- *Pass-oriented*: DeepSeek-V3.2, DeepSeek-V3.2-Thinking, and GPT-5.1-Codex exhibit substantially lower precision ($\leq 48\%$), with recall below unit test scores, indicating a regeneration-heavy behavior that relies on broad rewrites.

These results highlight the necessity of edit-level evaluation for distinguishing targeted debugging behavior from superficial pass-driven regeneration.

Negative correlation between unit score and precision. We further analyze model performance as the number of injected bugs increases ($k \in \{1, 2, 3, 4\}$), corresponding to increasing problem complexity. As shown in Figure 4, unit-test scores consistently decrease across all models as the number of bugs increases. At the same time, we observe an inverse trend for edit-level precision. Because models tend to over-edit, increasing the number of bugs raises the likelihood that a model modifies at least one necessary line, but also increases the amount of unnecessary edits, leading to lower precision overall. This trend is further supported by our analysis in Appendix B, which shows that precision degrades as buggy code length increases.

In contrast, recall primarily reflects debugging difficulty per bug, as it measures the fraction of bugs successfully addressed. Consistent with this interpretation, recall exhibits dataset-dependent behavior. On the API-focused BigCodeBench benchmark, where the difficulty of fixing individual bugs remains relatively stable, recall varies by less than 5% across bug counts from 1 to 4. On the algorithm-focused LiveCodeBench benchmark, where debugging difficulty increases with the number of injected bugs, recall shows a clear positive correlation with unit-test scores.

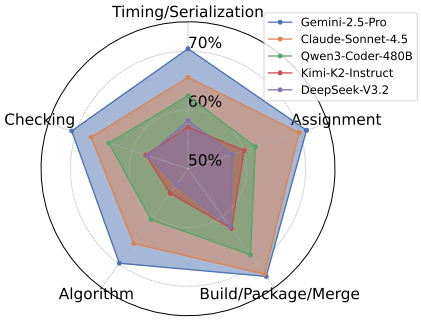


Figure 5: Bug-level recall by ODC category for five models spanning the four behavior groups above. The radial axis starts at 50%.

Recall across defect categories. Because bugs are generated per ODC category (§3), PDB also supports category-level analysis. As shown in Figure 5, with the exception of Gemini-2.5-Pro, whose recall is relatively uniform across categories, models achieve their highest recall on *Build/Package/Merge* defects. We hypothesize that this advantage arises from the higher prevalence of such defects, *e.g.*, invalid API calls and dependency-version conflicts, in model pretraining data, making them easier for models to recognize and repair. Appendix A further relates these categories to classical mutation operators (Table 5).

5.2 Iterative and agentic debugging

Next, we evaluate model behavior under *iterative* and *agentic* debugging settings. In *iterative debugging*, models produce an initial single-shot solution and are then allowed up to three revision attempts per problem. The process terminates early if a revision passes all unit tests. Models have



Figure 7: Comparison of model performance under minimal-debug and freeform prompting on a subset of PDB-SINGLE-FULL. Freeform prompting leads to substantial drops in precision and recall across all models, indicating prompt-level constraints are necessary to increase debugging precision.

access to their previous failed outputs, approximating an interactive debugging workflow commonly used by human programmers. In the *agentic* setting, models are likewise permitted up to three attempts, but additionally receive unit tests and execution error feedback at each step, resembling a simple agentic debugging pipeline with explicit external feedback. We evaluate on 500 instances randomly sampled from PDB-SINGLE, BigCodeBench sourced subset.

Functional gains without precision improvements. As shown in Figure 6, both iterative and agentic settings consistently improve unit-test scores and recall, indicating a higher likelihood of eventually producing functionally correct programs and resolving a larger fraction of bugs. However, these gains do not translate into improved edit-level precision. In most cases, precision remains unchanged or degrades relative to single-shot debugging. This pattern suggests that iterative interaction primarily improves correctness by expanding the scope of code modifications, rather than by refining or localizing edits toward minimal repairs.

Ineffective use of feedback in agentic debugging. Despite direct access to unit tests and execution feedback, most models fail to leverage this information to improve edit-level behavior in the agentic setting (Figure 6). In particular, agentic debugging often underperforms iterative debugging in precision, suggesting that additional feedback may exacerbate regeneration-oriented behaviors. Rather than supporting fault localization, test outcomes and error messages are frequently treated as coarse success signals that trigger further broad rewrites. These results indicate that access to feedback alone is insufficient to induce edit-aware debugging.

Regeneration persists even in Claude-Code. We observe similar trends in Claude-Code, which achieves the highest precision among agentic methods but still attains only approximately 50% precision. As shown in Figure 6, this result indicates that even more sophisticated, end-to-end agentic systems largely rely on regeneration rather than precise editing, reinforcing the conclusion that current debugging agents lack robust mechanisms for localized, minimal code repair.

5.3 Multi-line bug extension

To test whether the precision gap observed on PDB-SINGLE generalizes to multi-line bugs, we evaluate the three generator models and the most recent models on PDB-WILD. Table 1 shows that PDB-WILD is harder than PDB-SINGLE, but **increasing bug granularity does not close the precision gap between models**. The ranking is preserved: Claude-Opus-4.7 and Gemini-3.1-Pro again behave as relatively precise debuggers, with precision above 77%, while GPT-5.5 achieves a unit score above 84% and recall comparable to the top models (83%), but remains below 61% precision.

Table 1: Performance on PDB-WILD with tolerance $\epsilon = 1$. The same precision gap persists under multi-line and repo-level bugs.

Model	Precision	Recall	Unit (%)
Claude-Opus-4.7	**77.8**	83.4	75.8
Gemini-3.1-Pro-Preview	77.8	**85.6**	**85.1**
Claude-Sonnet-4.5	68.7	77.8	69.2
GPT-5.5	60.9	83.0	84.3
Gemini-2.5-Pro	58.3	70.5	68.8
Qwen3.6-plus	50.5	62.7	65.3
Kimi-K2.6	45.7	62.3	43.4
GPT-5.1-Codex	37.1	59.1	68.6

5.4 Prompting and data generation

For ablation study, we randomly sample 500 examples from PDB-SINGLE, to analyze how prompting and data generation strategies affect model debugging behavior.

Table 2: Rewriting ground truth consistently improves debugging precision, while using different generators makes buggy examples harder to solve.

Data	Precision	Recall	Unit (%)
Raw	73.0	83.1	76.2
Rewrite-Same-Gen	**76.5**	86.6	**76.4**
Rewrite-Different-Gen	75.8	**86.8**	74.8

Table 3: Precision, recall, and unit score comparison across source bug generation models for PDB-SINGLE, averaged over debug models.

BugGen Model	Count	Precision	Recall	Unit (%)
GPT-5.1-Codex	1809	61.5	83.1	78.8
Gemini-2.5-Pro	1937	58.1	75.7	71.0
Claude-Sonnet-4.5	1988	49.9	67.6	67.8

Freeform *vs.* minimal debugging. In our main experiments, models are explicitly instructed to perform debugging with minimal edits. To assess the impact of this constraint, we conduct an ablation in which models are instead prompted to debug freely, without any restriction on edit scope (see Appendix E for prompts). Figure 7 compares freeform to minimal-debug prompts. Across all evaluated models, freeform prompting results in a substantial drop in edit-level precision and bug-level recall. Even the strongest models, including Claude-Sonnet-4.5 and Qwen3-Coder-480B, achieve less than 60% precision under freeform prompting. Gemini-2.5-Pro exhibits a 40% absolute drop in precision, indicating that its apparent debugging precision largely stems from instruction following rather than intrinsic edit awareness. GPT-5.1-Codex performs particularly poorly under freeform prompts, failing to reach 20% precision. These results reinforce the regeneration behavior discussion, and demonstrate that *prompt-level constraints are necessary but insufficient*: while minimal-debug prompts reduce over-editing, they do not fundamentally change underlying model behavior.

Regeneration *vs.* contamination. Although regeneration dominates model behavior on PDB, an open question is whether this tendency is driven by data contamination, *i.e.*, overlap between solutions and model pretraining data. To disentangle these effects, we conduct two controlled analyses. First, we rewrite ground-truth solutions using rewriter models (Claude-Sonnet-4.5 or GPT-5.1-Codex), producing semantically equivalent but surface-diverse references. Second, we generate buggy programs using either the same model as the rewriter or a different generator model. As shown in Table 2, rewriting ground-truth solutions consistently makes debugging slightly easier, improving edit-level precision by 2.8-3.5% on average. In contrast, when buggy programs are generated by a different model from the rewriter, performance degrades, with unit-test pass rates dropping by up to 1.4%. This indicates that cross-model generation introduces additional variability that is more difficult for models to resolve. Taken together, these results suggest that while *data contamination* may marginally influence debugging performance, it does not account for the pervasive regeneration behavior observed on PDB.

Bug generator analysis. Table 3 reveals that bugs generated by GPT-5.1-Codex are consistently the easiest to debug, while those generated by Claude-Sonnet-4.5 are the hardest. This ordering is consistent across all evaluated metrics.

5.5 Metric verification and error analysis

We conduct a qualitative error analysis by manually inspecting two categories of failures: (1) cases passing unit tests with imperfect precision or recall, and (2) cases failing unit tests despite containing partially correct edits. This analysis assesses the robustness of our precision and recall metrics. We randomly selected 240 examples from these categories and derived the taxonomy described below; detailed examples are provided in Appendix D.

Passing unit tests with imperfect precision or recall. In this category, models successfully resolve the intended bug but introduce extraneous modifications.

Precision Analysis: In 83.5% of cases where unit tests pass, the recall score is also 1, but precision<1. We observe that 9.8% of edits add redundant guard checks, 66.8% modify correct code blocks, 13.7% apply correct but non-minimal edits, and 7.8% fully regenerate the solution. Notably, the remaining 1.9% of patches have low precision because they fix bugs that were missing from the ground-truth solutions. Thus, our *edit-level precision* accurately captures unnecessary edits. Further, to assess human agreement on precision, annotators with over five years of Python experience rate each patch on a 1–5 scale, with higher scores indicating more targeted and intent-preserving repairs. PDB precision strongly agrees with expert judgments, achieving Spearman $\rho = 0.72$.

Recall Analysis: Conversely, 16.5% of passing examples exhibit imperfect recall. In this scenario, we find that 70% of examples are functionally correct but evade recall detection due to over-editing or structural rewrites. Furthermore, 10% involve compounding bugs (*only 1.65% of all cases*), where an injected bug alters program logic such that other bugs change context, and another 20% arise because a single bug allows for multiple minimal correct fixes. This indicates that, provided bug independence is maintained during dataset creation, our *bug-level recall* is accurate for over 97.5% of all the data.

Failing unit tests with partially correct edits. In this category, models apply some correct edits but fail to fully resolve existing bugs or introduce new ones. We classify these failures into three types: (1) *Under-repair* (recall$<$1, precision$=$1): The model fixes some bugs without unnecessary edits but fails to apply all fixes (31.4%). (2) *Imprecise repair* (recall$<$1, precision$<$1): The model both misses fixes and introduces unnecessary or harmful edits (29.4%). (3) *Regressive repair* (recall$=$1): The model fixes all original bugs but introduces new errors that cause unit tests to fail; this accounts for the majority (39.2%). This gap highlights a silent reasoning challenge unique to debugging: models must understand program structure and preserve intent while restoring functional correctness, rather than merely generating working code.

6 Related Works

Debugging is a critical yet time-consuming stage of the software development lifecycle [Glass, 2002], making it a natural target for automation with LLMs. Recent systems increasingly emulate real-world debugging workflows, achieving improved performance through hierarchical multi-agent architectures [Lee et al., 2025, Bouzenia et al., 2025] and agent-based data synthesis with explicit communication [Yang et al., 2025b]. To evaluate such approaches, several general-purpose debugging benchmarks have been introduced, including those mined from historical bug-fixing commits [Tian et al., 2024] and those expanding coverage across programming languages [Liu et al., 2024]. Other benchmarks focus on specific dimensions of debugging, such as code editing [Guo et al., 2024], systematic analyses of automated bug fixing [Sobania et al., 2023], or broader coverage of debugging scenarios [Yuan et al., 2025, Huang et al., 2025, Chai et al., 2024].

However, existing benchmarks predominantly rely on unit-test–based evaluation, which rewards models equally for rewriting large portions of code and for making minimal, targeted fixes. In contrast, PDB introduces edit-level precision and bug-level recall, exposing fundamental shortcomings in current debugging systems and aligning evaluation more closely with real-world practices.

7 Discussion

We show that frontier LLMs often pass unit tests while remaining far from precise debugging, frequently relying on solution regeneration rather than targeted edits. By introducing PDB with edit-level precision and bug-level recall, we expose behaviors that unit-test-only evaluation misses, revealing a substantial gap between functional correctness and genuine fault localization. Results on PDB-SINGLE and PDB-WILD show that improving debugging requires rethinking both evaluation and post-training objectives. More broadly, because both older and recent models struggle to precisely fix self-generated bugs under PDB, the framework can serve not only as an evaluation benchmark but also as infrastructure for closing the training loop in self-improving coding models. Its precision-aware metrics provide reward signals focused on fault localization and edit minimality, while its plug-and-play pipeline enables scalable, controlled construction of debugging data from existing coding datasets.

Acknowledgments and Disclosure of Funding

We thank Yuqing Yang, Johnny Tian-Zheng Wei, Deqing Fu, Ting-Yun Chang, and Muru Zhang for reviewing early drafts of this work. We also thank Haozhe Lu for valuable discussions and brainstorming. WN was supported in part by the National Science Foundation under Grant No. CMMI-2427856. RJ was supported in part by the National Science Foundation under Grant No. IIS-2403436. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation.

References

M. AI. Kimi k2.6. https://www.kimi.com/blog/kimi-k2-6, 2026. URL https://www.kimi.com/blog/kimi-k2-6.

L. B. Allal, R. Li, D. Kocetkov, C. Mou, C. Akiki, C. M. Ferrandis, N. Muennighoff, M. Mishra, A. Gu, M. Dey, et al. Santacoder: don't reach for the stars! *arXiv preprint arXiv:2301.03988*, 2023.

J. H. Andrews, L. C. Briand, and Y. Labiche. Is mutation an appropriate tool for testing experiments? In *Proceedings of the 27th International Conference on Software Engineering*, 2005.

Anthropic. Claude sonnet 4.5. https://www.anthropic.com/news/claude-sonnet-4-5, 2025. URL https://www.anthropic.com/news/claude-sonnet-4-5.

Anthropic. Claude opus 4.7. https://www.anthropic.com/news/claude-opus-4-7, 2026. URL https://www.anthropic.com/news/claude-opus-4-7.

I. Bouzenia, P. Devanbu, and M. Pradel. Repairagent: An autonomous, llm-based agent for program repair. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, pages 2188–2200. IEEE, 2025.

L. Chai, S. Liu, J. Yang, Y. Yin, K. Jin, J. Liu, T. Sun, G. Zhang, C. Ren, H. Guo, et al. Mceval: Massively multilingual code evaluation. *arXiv preprint arXiv:2406.07436*, 2024.

M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. D. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

R. Chillarege, I. S. Bhandari, J. K. Chaar, M. J. Halliday, D. S. Moebus, B. K. Ray, and M.-Y. Wong. Orthogonal defect classification-a concept for in-process measurements. *IEEE Transactions on software Engineering*, 18(11):943–956, 1992.

H. Coles, T. Laurent, C. Henard, M. Papadakis, and A. Ventresque. PIT: a practical mutation testing tool for Java (demo). In *Proceedings of the 25th International Symposium on Software Testing and Analysis*, 2016.

G. Comanici, E. Bieber, M. Schaekermann, I. Pasupat, N. Sachdeva, I. Dhillon, M. Blistein, O. Ram, D. Zhang, E. Rosen, et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*, 2025.

Z. Dai, B. Chen, Z. Zhao, X. Tang, S. Wu, C. Yao, Z. Gao, and J. Chen. Less is more: Adaptive program repair with bug localization and preference learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025.

DeepSeek-AI. Deepseek-v3. 2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025.

W. Gao, X. Liu, F. Wang, D. Lu, and J. Yin. Decoding memories: An efficient pipeline for self-consistency hallucination detection. *arXiv preprint arXiv:2508.21228*, 2025.

R. L. Glass. *Facts and fallacies of software engineering*. Addison-Wesley Professional, 2002.

Google. Gemini 3.1 pro. https://deepmind.google/models/model-cards/gemini-3-1-pro/, 2026. URL https://deepmind.google/models/model-cards/gemini-3-1-pro/.

J. Guo, Z. Li, X. Liu, K. Ma, T. Zheng, Z. Yu, D. Pan, Y. Li, R. Liu, Y. Wang, et al. Codeeditorbench: Evaluating code editing capability of large language models. *arXiv preprint arXiv:2404.03543*, 2024.

D. Hendrycks, S. Basart, S. Kadavath, M. Mazeika, A. Arora, E. Guo, C. Burns, S. Puranik, H. He, D. Song, et al. Measuring coding challenge competence with apps. *arXiv preprint arXiv:2105.09938*, 2021.

M.-C. Hsueh, T. K. Tsai, and R. K. Iyer. Fault injection techniques and tools. *Computer*, 1997.

G. Hu, A. Khan, C. Zimmer, M. J. Brim, and F. Mueller. Fitcache: A transparent drop-in framework for multi-tier caching to accelerate distributed deep learning workloads. In *2026 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 458–472. IEEE, 2026.

J. Huang, X. Feng, Q. Chen, H. Zhao, Z. Cheng, J. Bai, J. Zhou, M. Li, and L. Qin. MLDebugging: Towards benchmarking code debugging across multi-library scenarios. In W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, editors, *Findings of the Association for Computational Linguistics: ACL 2025*, pages 5866–5879, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-256-5. doi: 10.18653/v1/2025.findings-acl.305. URL https://aclanthology.org/2025.findings-acl.305/.

B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Lu, et al. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.

N. T. Islam, J. Khoury, A. Seong, M. B. Karkevandi, G. D. L. T. Parra, E. Bou-Harb, and P. Najafirad. Llm-powered code vulnerability repair with reinforcement learning and semantic reward. *arXiv preprint arXiv:2401.03374*, 2024.

N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.

Y. Jia and M. Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 2011.

C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *The twelfth international conference on learning representations*, 2023.

M. Jin, S. Shahriar, M. Tufano, X. Shi, S. Lu, N. Sundaresan, and A. Svyatkovskiy. Inferfix: End-to-end program repair with llms. In *Proceedings of the 31st ACM joint european software engineering conference and symposium on the foundations of software engineering*, pages 1646–1656, 2023.

R. Just, D. Jalali, and M. D. Ernst. Defects4j: A database of existing faults to enable controlled testing studies for java programs. In *Proceedings of the 2014 international symposium on software testing and analysis*, pages 437–440, 2014.

Kimi. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*, 2025.

S. Kulal, P. Pasupat, K. Chandra, M. Lee, O. Padon, A. Aiken, and P. S. Liang. Spoc: Search-based pseudocode to code. *Advances in Neural Information Processing Systems*, 32, 2019.

C. Le Goues, M. Pradel, and A. Roychoudhury. Automated program repair. *Communications of the ACM*, 2019.

C. Lee, C. S. Xia, L. Yang, J.-t. Huang, Z. Zhu, L. Zhang, and M. R. Lyu. Unidebugger: Hierarchical multi-agent framework for unified software debugging. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 18248–18277, 2025.

R. Li, L. B. Allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, M. Marone, C. Akiki, J. Li, J. Chim, et al. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161*, 2023.

Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago, et al. Competition-level code generation with alphacode. *Science*, 378(6624): 1092–1097, 2022.

F. Liu, Y. Liu, L. Shi, Z. Yang, L. Zhang, X. Lian, Z. Li, and Y. Ma. Beyond functional correctness: Exploring hallucinations in llm-generated code. *IEEE Transactions on Software Engineering*, 2026.

S. Liu, L. Chai, J. Yang, J. Shi, H. Zhu, L. Wang, K. Jin, W. Zhang, H. Zhu, S. Guo, et al. Mdeval: Massively multilingual code debugging. *arXiv preprint arXiv:2411.02310*, 2024.

Z. Ma, T. Zhang, J. Liu, W. Zhang, M. Luo, S. Zhang, and K. Chen. Rethinking verification for llm code generation: From generation to testing. *Advances in Neural Information Processing Systems*, 38:137549–137582, 2026.

A. Mohsin, H. Janicke, A. Wood, I. H. Sarker, L. Maglaras, and N. Janjua. Can we trust large language models generated code? a framework for in-context learning, security patterns, and code evaluations across diverse llms. *arXiv preprint arXiv:2406.12513*, 2024.

A. Ni, S. Iyer, D. Radev, V. Stoyanov, W.-t. Yih, S. Wang, and X. V. Lin. Lever: Learning to verify language-to-code generation with execution. In *International Conference on Machine Learning*, pages 26106–26128. PMLR, 2023.

E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong. Codegen: An open large language model for code with multi-turn program synthesis. *arXiv preprint arXiv:2203.13474*, 2022.

M. Ojdanic, A. Garg, A. Khanfir, R. Degiovanni, M. Papadakis, and Y. Le Traon. Syntactic versus semantic similarity of artificial and real faults in mutation testing studies. *IEEE Transactions on Software Engineering*, 2023.

OpenAI. Gpt-5.1 codex. https://openai.com/index/gpt-5-1-for-developers/, 2025a. URL `https://openai.com/index/gpt-5-1-for-developers/`.

OpenAI. Gpt-5.2 codex. https://openai.com/index/introducing-gpt-5-2-codex/, 2025b. URL `https://openai.com/index/introducing-gpt-5-2-codex/`.

OpenAI. Gpt-5.5. https://openai.com/index/introducing-gpt-5-5/, 2026. URL `https://openai.com/index/introducing-gpt-5-5/`.

L. Phan, H. Tran, D. Le, H. Nguyen, J. Annibal, A. Peltekian, and Y. Ye. Cotext: Multi-task learning with code-text transformer. In *Proceedings of the 1st Workshop on Natural Language Processing for Programming (NLP4Prog 2021)*, pages 40–47, 2021.

Qwen. Qwen3 technical report, 2025. URL `https://arxiv.org/abs/2505.09388`.

Qwen Team. Qwen3.6-Plus: Towards real world agents, April 2026. URL `https://qwen.ai/blog?id=qwen3.6`.

B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez, et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

M. Sevenhuijsen, K. Etemadi, and M. Nyberg. Vecogen: Automating generation of formally verified c code with large language models. In *2025 IEEE/ACM 13th International Conference on Formal Methods in Software Engineering (FormaliSE)*, pages 101–112. IEEE, 2025.

M. Singhal, T. Aggarwal, A. Awasthi, N. Natarajan, and A. Kanade. Nofuneval: Funny how code lms falter on requirements beyond functional correctness. *arXiv preprint arXiv:2401.15963*, 2024.

D. Sobania, M. Briesch, C. Hanna, and J. Petke. An analysis of the automatic bug fixing performance of chatgpt. In *2023 IEEE/ACM International Workshop on Automated Program Repair (APR)*, pages 23–30. IEEE, 2023.

F. Tambon, A. Moradi-Dakhel, A. Nikanjam, F. Khomh, M. C. Desmarais, and G. Antoniol. Bugs in large language models generated code: An empirical study. *Empirical Software Engineering*, 30 (3):65, 2025.

R. Tian, Y. Ye, Y. Qin, X. Cong, Y. Lin, Y. Pan, Y. Wu, H. Haotian, L. Weichuan, Z. Liu, et al. Debugbench: Evaluating debugging capability of large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 4173–4198, 2024.

Y. Wang, H. Le, A. Gotmare, N. Bui, J. Li, and S. Hoi. Codet5+: Open code large language models for code understanding and generation. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pages 1069–1088, 2023.

Y. Wei, C. S. Xia, and L. Zhang. Copiloting the copilots: Fusing large language models with completion engines for automated program repair. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 172–184, 2023.

xAI. Grok code fast 1. https://x.ai/news/grok-code-fast-1, 2025. URL `https://x.ai/news/grok-code-fast-1`.

C. S. Xia and L. Zhang. Automated program repair via conversation: Fixing 162 out of 337 bugs for $0.42 each using chatgpt. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 819–831, 2024.

J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.

J. Yang, K. Lieret, C. E. Jimenez, A. Wettig, K. Khandpur, Y. Zhang, B. Hui, O. Press, L. Schmidt, and D. Yang. Swe-smith: Scaling data for software engineering agents. *arXiv preprint arXiv:2504.21798*, 2025a.

W. Yang, H. Wang, Z. Liu, X. Li, Y. Yan, S. Wang, Y. Gu, M. Yu, Z. Liu, and G. Yu. Coast: Enhancing the code debugging ability of llms through communicative agent based data synthesis. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 2570–2585, 2025b.

M. Yasunaga and P. Liang. Break-it-fix-it: Unsupervised learning for program repair. In *International conference on machine learning*, pages 11941–11952. PMLR, 2021.

P. Yin, B. Deng, E. Chen, B. Vasilescu, and G. Neubig. Learning to mine aligned code and natural language pairs from stack overflow. In *Proceedings of the 15th international conference on mining software repositories*, pages 476–486, 2018.

X. Yuan, M. M. Moss, C. E. Feghali, C. Singh, D. Moldavskaya, D. MacPhee, L. Caccia, M. Pereira, M. Kim, A. Sordoni, et al. debug-gym: A text-based environment for interactive debugging. *arXiv preprint arXiv:2503.21557*, 2025.

L. Zhang, W. Chen, L. Zhong, L. Peng, Z. Wang, and J. Shang. Memorize or generalize? evaluating llm code generation with code rewriting. *arXiv preprint arXiv:2503.02296*, 2025.

Q. Zhang, C. Fang, Y. Ma, W. Sun, and Z. Chen. A survey of learning-based automated program repair. *ACM Transactions on Software Engineering and Methodology*, 33(2):1–69, 2023a.

Q. Zhang, T. Zhang, J. Zhai, C. Fang, B. Yu, W. Sun, and Z. Chen. A critical review of large language model on software engineering: An example from chatgpt and automated program repair. *arXiv preprint arXiv:2310.08879*, 2023b.

L. Zhong, Z. Wang, and J. Shang. Debug like a human: A large language model debugger via verifying runtime execution step by step. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 851–870, 2024.

M. Zhong, X. Zhou, T.-Y. Chang, Q. Wang, N. Xu, X. Si, D. Garrette, S. Upadhyay, J. Liu, J. Han, et al. Vibe checker: Aligning code evaluation with human preference. *arXiv preprint arXiv:2510.07315*, 2025.

T. Y. Zhuo, M. C. Vu, J. Chim, H. Hu, W. Yu, R. Widyasari, I. N. B. Yusuf, H. Zhan, J. He, I. Paul, et al. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. *arXiv preprint arXiv:2406.15877*, 2024.

Appendix

A Additional Related Works **16**

B Additional Experiments **19**

B.1 Results on the full PDB-SINGLE-FULL pool 19

B.2 Buggy code length scaling 20

B.3 Source rewriting and cross-model bug generation 20

B.4 Real-world debugging on DEBUGBENCH 21

B.5 Unit-test oracle reliability 21

B.6 Multi-line bugs and post-release models on PDB-WILD 22

C Algorithm on Precision and Recall **23**

D Examples of Debugging Categories **26**

D.1 Failure modes when unit tests pass 26

D.2 Failure modes when unit tests fail 28

E Prompt templates **36**

F Checklist Information **45**

G Limitation **46**

A Additional Related Works

We discuss additional related work in the broader context of code generation and debugging.

Code Generation. The capability of LLMs in code generation has been transforming both academia and industry. Beginning with seminal models like Codex [Chen et al., 2021], the field has rapidly advanced with the introduction of dozens of powerful code-centric models including Code Llama [Roziere et al., 2023], StarCoder [Li et al., 2023, Allal et al., 2023], CodeGen [Nijkamp et al., 2022], CodeT5+ [Wang et al., 2023], Qwen Coder Series [Hui et al., 2024, Qwen, 2025], and more recent GPT-5.1 and 5.2 codex OpenAI [2025a,b]. These models, trained on vast web-scale datasets of code, excel at synthesizing end-to-end programs from natural language prompts. To evaluate their capabilities, numerous benchmarks have been established, ranging from function-level synthesis tasks like HumanEval [Chen et al., 2021] and CoNaLa [Yin et al., 2018], to more complex challenges including APPS [Hendrycks et al., 2021], CodeContests [Li et al., 2022], SPOC [Kulal et al., 2019], and BigCodeBench [Zhuo et al., 2024]. Beyond functional correctness, works like Vibe Checker [Zhong et al., 2025] and NoFunEval [Singhal et al., 2024] target the evaluation of models' non-functional instruction-following abilities. Recent agent-based systems like SWE-Agent [Yang et al., 2024] and AutoGen [Sevenhuijsen et al., 2025] demonstrate the potential of LLMs in autonomous development workflows. Recently, Zhang et al. [2025] examined memorization effects in LLM-based code generation. We evaluated this hypothesis through targeted rewriting experiments (Table 2) and found that memorization is not the root cause of regenerator-style behavior in debugging.

Debugging Frameworks. As a critical and often time-consuming task, debugging has naturally emerged as another target for automation using LLMs. This need is further magnified by the fact that code generation models themselves are a significant source of buggy and potentially vulnerable code [Ni et al., 2023, Jin et al., 2023, Mohsin et al., 2024, Tambon et al., 2025, Liu et al., 2026, Gao et al., 2025]. Consequently, a spectrum of approaches have been proposed to leverage these models for program repair. Before LLMs, automated program repair was predominantly search-based, with optimization criteria that include the size of the patch [Le Goues et al., 2019]; recent work revisits patch minimality directly, *e.g.*, AdaPR combines bug localization with preference learning to generate patches with minimal modifications [Dai et al., 2025]. Early work like Break-It-Fix-It [Yasunaga and Liang, 2021] [Zhang et al., 2023a] introduced unsupervised learning for program repair, while CoText [Phan et al., 2021] explored multi-task learning with code-text transformers. Recent systems emulate real-world debugging workflows through sophisticated agent architectures: FixAgent [Lee et al., 2025] employs hierarchical multi-agent frameworks, RepairAgent [Bouzenia et al., 2025] demonstrates autonomous repair capabilities, and COAST [Yang et al., 2025b] enhances debugging through communicative agent-based data synthesis. These approaches utilize techniques ranging from zero-shot prompting to multi-turn conversational agents [Chen et al., 2021, Wei et al., 2023, Xia and Zhang, 2024, Zhong et al., 2024, Islam et al., 2024, Hu et al., 2026].

Debugging Evaluation To evaluate the performance of these LLM debugging approaches, a handful of benchmarks have been established. Early work like Defects4J [Just et al., 2014] provided curated bug datasets from real-world Java projects, while recent benchmarks have adapted to the LLM era. Tian et al. [2024] create debugging scenarios by mining historical bug-fixing commits. Liu et al. [2024] curates multi-lingual code repair tasks spanning Python, Java, and JavaScript, while Jain et al. [2024] mitigates data contamination by using live programming contests. Specialized benchmarks like CodeEditorBench [Guo et al., 2024] focus on code editing capabilities, and analyses like Sobania et al. [2023] examine automatic bug fixing performance on existing datasets. A common limitation of these benchmarks, however, is their reliance on a simple, binary pass/fail metric on test cases [Zhang et al., 2023b]. Such coarse-grained evaluation is insufficient, as it cannot distinguish between a minimal, targeted fix and a complete code regeneration that merely passes the tests—a distinction crucial for understanding whether models truly comprehend debugging or simply regenerate working solutions. In contrast, our proposed PDB disentangles debugging from code generation, introducing fine-grained evaluation metrics that assess not only functional correctness but also the precision, minimality, and human-like nature of code repairs, better reflecting real-world debugging practices where understanding and fixing the root cause is valued over wholesale replacement.

Mutation Testing and Fault Injection. Deliberately introducing faults into programs has a long history in software testing, under goals that differ from ours. *Fault seeding* inserts known faults to

evaluate a testing or quality-assurance process, and *mutation testing*, its most widely studied form, applies small syntactic changes called *mutation operators* to produce faulty program variants (*mutants*) and measures test-suite adequacy by the fraction of mutants the tests detect [Jia and Harman, 2011]. Whether mutants are valid substitutes for real faults has been studied extensively [Andrews et al., 2005, Ojdanic et al., 2023], and practical tools such as PIT for Java [Coles et al., 2016] implement standard operators, *e.g.*, relational and arithmetic operator replacement. *Fault injection*, in turn, introduces faults or fault conditions into a running system to study its fault-tolerance mechanisms [Hsueh et al., 1997]. Bug injection in PDB shares the mechanics of fault seeding but not its purpose: following prior debugging benchmarks [Tian et al., 2024, Liu et al., 2024], we take the unit tests as the correctness oracle and use injected bugs as debugging tasks for a model to fix, rather than as probes of the tests themselves. This difference also sidesteps the *equivalent-mutant problem*, in which a mutation leaves program behavior unchanged so that no test can detect it, which is undecidable in general [Jia and Harman, 2011]: PDB retains an injected bug only if the unit tests fail on it, and the atomicity filter further requires every partial fix to fail (§3), so every retained bug is observably different from the ground-truth program.

We generate bugs with LLMs rather than mutation operators for three reasons. First, the generator is conditioned on the natural-language task description in addition to the code, so it produces faults that are plausible for the task being solved, such as a boundary mistake that a programmer implementing that specification might make, rather than uniformly sampled operator applications. Second, operator-style perturbations yield bugs that are too easy to separate models: in an early pilot, frontier models reached at least 90% on both unit-test pass rate and precision on such perturbations, leaving little room for analysis or model comparison. Third, PDB-WILD requires contiguous multi-line bugs whose partial fixes still fail the tests, which are semantically coherent faults that operator-based seeding does not generate. Table 5 relates common mutation operators to the defect taxonomy below.

Orthogonal Defect Classification. The Orthogonal Defect Classification (ODC) framework [Chillarege et al., 1992] partitions software defects into eight semantically disjoint defect types—*Function*, *Interface*, *Checking*, *Assignment*, *Timing/Serialization*, *Build/Package/Merge*, *Documentation*, and *Algorithm*—so that any given bug can be assigned to exactly one type without overlap. *Function* errors affect significant capability and require a formal design change, and *Documentation* errors concern publications rather than code, so neither can be synthesized as a line-level edit inside a static seed program; we fold errors in interacting with external libraries into *Build/Package/Merge* (Table 4) instead of a separate *Interface* type. We retain the five types that admit purely code-level expression—**Assignment**, **Checking**, **Algorithm**, **Build/Package/Merge**, and **Timing/Serialization**—and use them as the in-context taxonomy that conditions the bug-injection generator. We use ODC as a taxonomy rather than as an injection mechanism: its categories are language-agnostic, so adapting PDB to another language only requires new per-category in-context examples, and they enable the category-level analysis in §5 (Figure 5).

Table 4 expands each retained ODC category into the concrete sub-patterns we surface to the generator. *Assignment* captures Python-specific reference and scoping pitfalls (mutable defaults, late binding in closures, shadowed built-ins); *Checking* covers boundary, negation, and validation faults at branch points; *Algorithm* groups deeper logical and arithmetic errors that violate program invariants; *Build/Package/Merge* reflects API-misuse and dependency-version faults that surface when a program meets its environment; and *Timing/Serialization* encodes serialization and async-blocking issues that are easy to introduce but hard to diagnose by inspection. Anchoring the generator to this taxonomy avoids degenerate failure modes where the model converges on a single defect family (typically *Checking*) and instead yields broad coverage across the precision/recall regimes our evaluation later stratifies.

ODC Categories *vs.* Mutation Operators. Table 5 maps the default mutation operators of PIT and mutmut to ODC categories and to the closest PDB sub-category in Table 4. The operators concentrate on *Checking*, through boundary, negation, and membership flips, and on arithmetic within *Algorithm*; together they reach only 10 of the 23 distinct sub-categories. The remaining 13, including Python-specific reference and scoping pitfalls under *Assignment* and environment-level faults under *Build/Package/Merge* and *Timing/Serialization*, have no default operator counterpart. Conditioning an LLM generator on the ODC taxonomy therefore covers defect types that operator-based seeding does not reach, while still subsuming the operator-style faults within *Checking* and *Algorithm*.

Table 4: ODC-style taxonomy of common programming defects with summarized descriptions. These are used as in-context examples.

ODC Category	Sub-category	Brief Description
Assignment	Mutability Trap	Mutable default arguments cause unintended shared state across calls.
	Late Binding in Closures	Loop variables captured by reference, yielding unexpected final values.
	List Multiplication Surprise	List multiplication creates multiple references to the same inner object.
	Built-in Shadowing	Assigning to names like `list` or `sum` hides built-ins.
	Variable Shadowing	Inner-scope variables obscure outer-scope references.
	Name Error	Variable is used before being assigned or defined.
Checking	Off-by-One Error	Boundary condition is shifted by exactly one element or unit.
	Negation Error	Boolean condition is logically inverted.
	Missing or Incomplete Checks	Absent validation leads to runtime errors (e.g., KeyError, TypeError).
	Overwriting Built-in Names	Built-in identifiers are reassigned, breaking later function calls.
	Variable Shadowing	Confusing variable scope leads to incorrect condition evaluation.
	Chained Boolean Comparison Logic	Misparsed chained comparisons yield unintended logic.
	Implicit Boolean Conversion	Empty collections and `None` are conflated in boolean context.
	Membership Logic Flaws	Misunderstanding how membership tests behave for data types.
Algorithm	Wrong Math Expression	Mathematical formula or operands are incorrectly specified.
	Modifying While Iterating	Collection is altered during iteration, skipping or misprocessing elements.
	Function Algorithm Misunderstanding	Function behavior is misunderstood (e.g., substring vs. set semantics).
	Function Argument Misunderstanding	Incorrect interpretation of function arguments or defaults.
	Infinite Loop / Recursion	Termination condition is missing or unreachable.
	Other Logical Errors	Deeper algorithmic invariants are violated during execution.
Build/Package/Merge	Invalid API Call	Method is invoked on an unsupported data type or abstraction.
	Dependency Version Conflicts	Code relies on APIs removed or changed across library versions.
Timing/Serialization	Serialization Issue	Non-serializable objects are passed to pickle or JSON encoders.
	Async Blocking	Blocking calls inside async code stall the event loop.

Table 5: Default mutation operators of PIT [Coles et al., 2016] and mutmut mapped to ODC categories and to the closest PDB sub-category in Table 4. Examples are written in Python. Operators concentrate on *Checking* and on arithmetic within *Algorithm*; the sub-categories in the last row have no default operator counterpart.

Mutation Operator	Tool	Example	ODC Category	Closest PDB Sub-category
Relational boundary replacement	PIT, mutmut	`i < n` → `i <= n`	**Checking**	Off-by-One Error
Numeric constant change	mutmut	`nums[0]` → `nums[1]`	**Checking**	Off-by-One Error
Conditional negation	PIT, mutmut	`x == y` → `x != y`; `True` → `False`	**Checking**	Negation Error
Membership and identity swap	mutmut	`x in seen` → `x not in seen`; `is` → `is not`	**Checking**	Membership Logic Flaws; Negation Error
Logical connector replacement	mutmut	`a and b` → `a or b`	**Checking**	Chained Boolean Comparison Logic
Arithmetic and bitwise replacement	PIT, mutmut	`a + b` → `a - b`; `a & b` → `a | b`	**Algorithm**	Wrong Math Expression
Increment and sign change	PIT, mutmut	`i += 1` → `i -= 1`; `-x` → `+x`	**Algorithm**	Wrong Math Expression; Infinite Loop / Recursion
Argument replacement or removal	mutmut	`f(x, k)` → `f(None, k)`	**Algorithm**	Function Argument Misunderstanding
String-method swap	mutmut	`s.find(c)` → `s.rfind(c)`	**Algorithm**	Function Algorithm Misunderstanding
Control-flow change or call removal	PIT, mutmut	`continue` → `break`; `nums.sort()` removed	**Algorithm**	Other Logical Errors
Copy-depth swap	mutmut	`deepcopy(grid)` → `copy(grid)`	**Assignment**	List Multiplication Surprise (shared inner objects)
Value replacement	PIT, mutmut	`return total` → `return None`	**Assignment**	—
No default operator	—	—	**Assignment**, **Checking**, **Algorithm**, **Build/Package/Merge**, **Timing/Serialization**	Mutability Trap, Late Binding in Closures, Built-in Shadowing, Variable Shadowing, Name Error, Missing or Incomplete Checks, Overwriting Built-in Names, Implicit Boolean Conversion, Modifying While Iterating, Invalid API Call, Dependency Version Conflicts, Serialization Issue, Async Blocking

B Additional Experiments

§B.1 shows that the model ranking observed on PDB-SINGLE persists on the unfiltered PDB-SINGLE-FULL pool. §B.2–§B.3 report ablations that isolate the contribution of buggy code length and source rewriting to the headline precision/recall numbers; the defect-category breakdown appears in §5. §B.4–§B.6 test transfer to a human-curated benchmark and a contemporaneous bug generator, addressing the concern that our findings are confined to a particular synthetic distribution.

B.1 Results on the full PDB-SINGLE-FULL pool

Table 6: Precision, recall, and unit score on the PDB-SINGLE-FULL set. Blue indicates better performance, while red indicates worse.

Model	Precision	Recall	Unit (%)
Claude-Sonnet-4.5	**78.1±0.7**	85.7±0.6	81.9±0.9
Gemini-2.5-Pro	77.9±0.7	**87.5±0.6**	83.8±0.8
Qwen3-Coder-480B	73.5±0.8	82.4±0.7	77.4±0.9
Kimi-K2-Instruct	65.8±0.8	78.8±0.7	73.0±1.0
Grok-Code-Fast	63.8±0.9	73.2±0.8	67.1±1.1
Kimi-K2-Thinking	61.3±0.8	81.2±0.7	80.8±0.9
DeepSeek-V3.2	58.6±0.9	76.2±0.8	78.2±0.9
DeepSeek-V3.2-Thinking	56.0±0.9	77.5±0.8	**84.7±0.8**
GPT-5.1-Codex	50.3±0.8	77.8±0.8	82.0±0.9

Table 6 reports the same nine models on PDB-SINGLE-FULL, the unfiltered superset from which PDB-SINGLE is sampled. All three metrics are 4–8% higher than on PDB-SINGLE (Table 3b), as expected: the difficulty filter that defines PDB-SINGLE removes easy examples that would otherwise lift averages. Crucially, the model ordering is preserved: Claude-Sonnet-4.5 and Gemini-2.5-Pro retain the highest precision (78.1% and 77.9%), Qwen3-Coder-480B remains the strongest mid-tier debugger, and GPT-5.1-Codex remains the weakest on precision (50.3%) despite a competitive unit-test pass rate. The stability of this ranking across the full and filtered pools indicates that the precision gap reported in the main paper is a property of model behaviour, not an artefact of the difficulty subset.

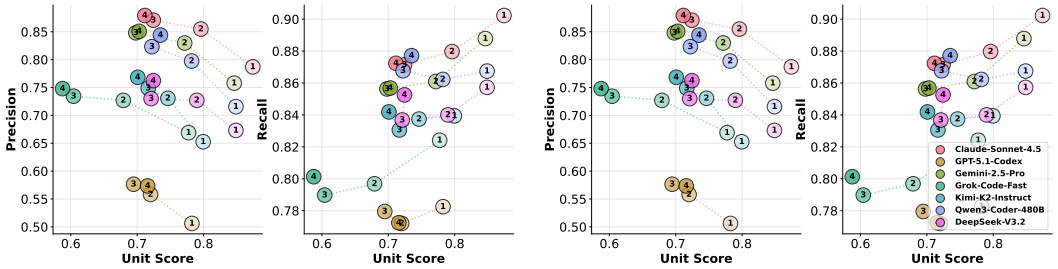


Figure 8: Per-bug-count breakdown of precision, recall, and unit score on the full PDB-SINGLE-FULL pool drawn from BigCodeBench (left) and LiveCodeBench (right). Bug counts $k \in \{1, 2, 3, 4\}$ are annotated next to each marker. Larger k pulls precision down monotonically while unit-test pass rate decays more slowly; recall is essentially flat on BigCodeBench and increases with k on LiveCodeBench.

Figure 8 stratifies the same metrics by the number of injected bugs k on each source benchmark. The negative correlation between unit score and precision sharpens compared to the filtered PDB-SINGLE version (Figure 4): every additional bug increases the chance that the model touches at least one needed line, but it also magnifies its over-editing tendency. The recall trend separates the two source benchmarks. On the API-heavy BigCodeBench, per-bug fix difficulty is roughly stationary, so recall stays within a 5% band across k. On the algorithm-heavy LiveCodeBench, individual bugs become harder when the program is already broken in multiple places, and recall tracks the unit-test pass rate upward as k grows. Both observations were already visible on PDB-SINGLE and become

more pronounced on the larger PDB-SINGLE-FULL pool, indicating they are systematic rather than sample-specific.

B.2 Buggy code length scaling

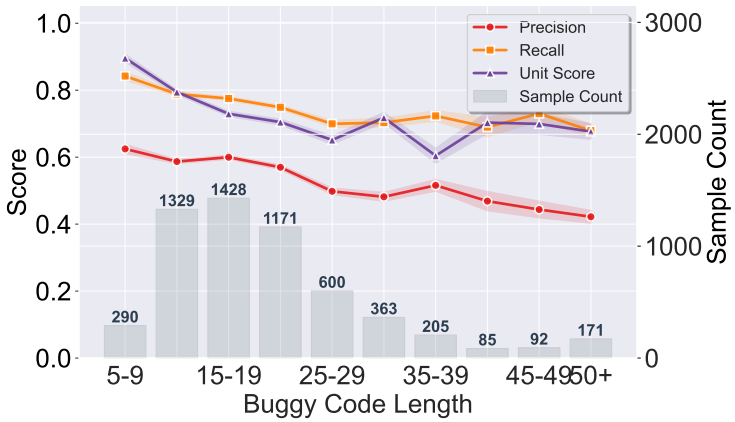


Figure 9: Model-averaged precision, recall, and unit-test pass rate on PDB-SINGLE, plotted as a function of buggy code length (lines of code). All three metrics decay together as the program grows.

Figure 9 averages each metric across the nine evaluated models and bins the results by the length of the buggy program. Longer programs hurt every metric simultaneously: the model first has to localize the bug among more candidate lines, then has to apply a precise edit without disturbing the surrounding code. Because the curves move in lockstep, the precision drop is not explained by tasks becoming purely harder to solve; if that were the case, recall and unit score would degrade while precision held steady. Instead, longer programs invite proportionally more spurious edits, which is consistent with the regeneration-style behaviour identified in the main text.

B.3 Source rewriting and cross-model bug generation

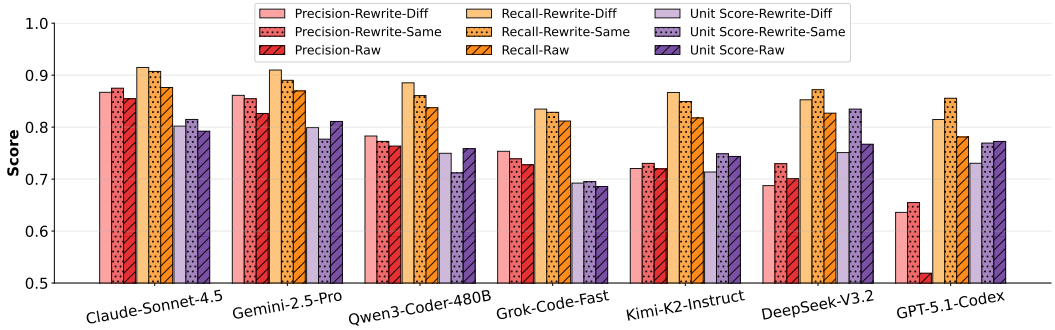


Figure 10: Per-model precision, recall, and unit-test pass rate on PDB-SINGLE when ground-truth solutions are rewritten by the same generator that introduced the bugs (*Same-Gen*) versus a different generator (*Different-Gen*), compared against the unmodified raw data.

Figure 10 expands the aggregate rewrite ablation in Table 2 into a per-model view. The aggregate finding—rewriting raises precision by 2.8–3.5% on average, while cross-model bug generation lowers unit pass rate by up to 1.4%—holds for every model individually, but the magnitude varies. Models with the strongest baseline precision (Claude-Sonnet-4.5, Gemini-2.5-Pro) gain the least from rewriting, indicating that they were not relying on incidental string overlap with their own pretraining data; conversely, the weakest models on precision (GPT-5.1-Codex, DeepSeek-V3.2) gain disproportionately less than they lose under cross-model generation, the opposite of what a memorization-driven account would predict. The breakdown therefore reinforces the conclusion in §5 that contamination is at most a secondary contributor to the observed precision gap.

Table 7: Model performance on DebugBench, evaluated with the PDB framework. The qualitative pattern observed on PDB-SINGLE and PDB-WILD persists: high unit-test pass rates coexist with substantially lower edit-level precision.

Model	Precision	Recall	Unit (%)
Claude-Sonnet-4.5	78.4	87.3	87.5
Gemini-2.5-Pro	**79.4**	**89.4**	85.0
GPT-5.1-Codex	61.9	74.0	**90.0**

B.4 Real-world debugging on DEBUGBENCH

Because PDB constructs debugging instances by injecting verified bugs, a natural concern is whether the observed precision gap is an artifact of synthetic bug generation. We examine this concern from two complementary perspectives: whether the same model behavior transfers to naturally occurring bugs, and whether PDB instances are distinguishable from real bugs at the instance level.

First, we extend the PDB evaluation framework to DEBUGBENCH [Tian et al., 2024], a human-validated synthetic debugging benchmark, to test whether the precision/recall pattern transfers. DEBUGBENCH does not annotate bug counts, so we approximate them through edit structure: we keep only examples whose ground-truth fixes form contiguous edit blocks under the same stride constraint $s = 5$ used for PDB-WILD, treating each block as one bug. The filter yields a subset of 40 examples, each containing exactly one bug under the stride-5 grouping, sampled uniformly at random from the qualifying pool.

Table 7 reports three frontier models evaluated under our protocol on this subset. Absolute scores are higher than on PDB-SINGLE or PDB-WILD, reflecting that DEBUGBENCH bugs are individually easier (one block per example, drawn from real fixes that human reviewers already accepted). Even so, the qualitative pattern persists: GPT-5.1-Codex obtains the highest unit-test pass rate (90.0%) but the lowest precision (61.9%), with a recall–precision gap of roughly 12 points; Claude-Sonnet-4.5 and Gemini-2.5-Pro again sit in the precise-debugger regime with precision approaching 80%. The same recall–precision dissociation that motivates PDB therefore reappears on a benchmark that we did not construct, indicating that the gap is a property of how current models repair code rather than of any specific synthetic injection pipeline.

We further conduct a controlled human study. We sample 100 PDB instances from the LiveCodeBench subset [Jain et al., 2024], evenly stratified across bug counts $k \in \{1, 2, 3, 4\}$, and 100 DEBUGBENCH instances matched by bug count and problem domain. Expert judges with more than five years of Python experience are shown only the task description and buggy program, with the instances randomly shuffled and their provenance concealed. For each instance, judges predict whether the bug occurred naturally or was injected by an LLM. Their classification accuracy is 55.0% (Wilson 95% CI: 48.1%–61.7%), which is not significantly different from the 50% chance level under a two-sided exact binomial test ($p = 0.179$). We therefore find no evidence that experts can reliably distinguish PDB bugs from naturally occurring bugs.

B.5 Unit-test oracle reliability

First, the source benchmarks were themselves built with test adequacy as an explicit design goal. BigCodeBenchZhuo et al. [2024] reports an average branch coverage of 99% per task, with multiple test cases exercising API-call behavior. LiveCodeBench Jain et al. [2024] draws from live competitive-programming contests, whose tests are adversarially designed to eliminate incorrect solutions, and further augments them with generator-based adversarial tests. SWE-smithYang et al. [2025a] inherits the full test suites of real repositories and validates each instance via `FAIL_TO_PASS` tests (the bug must break them) and `PASS_TO_PASS` tests (the rest of the suite must still pass), so both bug detection and non-regression are checked against production test suites.

Second, the remaining risk is false positives at scoring time: a model patch that passes the tests while being actually incorrect. To quantify this, we randomly sampled 100 examples in which the model's patch passed the unit tests, and asked expert annotators with more than five years of Python experience to independently verify each patched program against the task description. 0 of 100

patches were found to be erroneous, indicating that test-passing is a reliable correctness signal on our benchmarks.

Third, we measure the false-positive rates from the patches on PDB-Single (LiveCodeBench) that pass the inherited tests but differ from the ground truth, the entire population in which a scoring false positive could occur. We sampled two disjoint batches of 108 patches (24 per model in total for the models reported in 3b) and submitted all 216 to LeetCode's official online judge, which runs the full hidden test suites. Results: 212/216 accepted; 4 rejected, a false-positive rate of 1.85%. The rejections were spread across four different models, with one rejection each for Gemini-2.5-Pro, Qwen3-Coder-480B, Kimi-K2-Thinking, and DeepSeek-V3.2. Note that in code completion tasks, even with human-LLM collaborative test generationMa et al. [2026], LiveCodeBench has a false-positive rate of 6.5% in a sample of 101 examples.

We hypothesize the rate is this low because debugging differs structurally from generation: candidate patches start from a nearly correct buggy program, so a weak suite only needs to catch the model's divergence from the reference, which is typically small unless the model substantially regenerates the program. Consistent with this, rejected patches have markedly lower precision than accepted ones (mean 0.25 vs. 0.45; 3 of 4 rejections below 0.5): heavily rewritten patches are exactly where hidden-test failures hide. We find this a striking additional argument for the precision metric itself: precision flags the patches that functional scoring may be unreliable.

B.6 Multi-line bugs and post-release models on PDB-WILD

A natural concern with PDB-SINGLE is that the headline numbers in Table 3b characterize models that may have observed the seed BCB/LCB programs during pretraining or post-training. Five of the strongest contemporary models, Claude-Opus-4.7 Anthropic [2026], Gemini-3.1-Pro Google [2026], GPT-5.5 OpenAI [2026], Qwen-3.6-Plus Qwen Team [2026], and Kimi-K2.6 AI [2026], were released after PDB-SINGLE became publicly available, so we cannot rule out that any improvement they show on the same data partly reflects contamination of the seed programs themselves.

To mitigate this concern, we use Claude-Opus-4.7, a contemporaneous frontier model that did not exist when the original generators ran, to synthesize multi-line, repository-scale bugs and assemble PDB-WILD. Table 1 reports the five post-release models alongside three of the original generators on PDB-WILD. The same precision deficit reappears: Claude-Opus-4.7 and Gemini-3.1-Pro lead with precision near 78%, GPT-5.5 attains a competitive unit pass rate (84.3%) but only 60.9% precision, and GPT-5.1-Codex collapses to 37.1% precision despite a 68.6% unit-test pass rate. Two implications follow. First, the precision gap is robust to the choice of bug generator and to whether the seed code was previously seen by the debugger. Second, because the PDB pipeline can be re-instantiated with any newer frontier generator, the benchmark naturally evolves with the model frontier rather than aging into the training data.

C Algorithm on Precision and Recall

We formalize the block-matching primitives used in the precision and recall metrics defined in §2. Algorithm 1 defines the matcher map, which pairs predicted edit blocks with ground-truth edit blocks; Algorithm 2 defines $\mathrm{essential}_{\mathcal{U}}$, which verifies each pairing with unit tests and computes the ϵ-relaxed essential edit size $(|\hat{E}_i|)_\epsilon$ used in Eq. (3). Both operate on edit blocks of any length, so they apply unchanged to the single-line bugs of PDB-SINGLE and the multi-line bugs of PDB-WILD.

The matcher map aligns predicted edit blocks to ground-truth edit blocks in two passes. Pass 1 accepts exact matches without testing: a predicted block that is identical to a ground-truth block, covering the same lines with the same edits. Matching whole blocks ensures that a partial fix inside a multi-line ground-truth block is never mistaken for an exact match. Pass 2 matches each remaining predicted block by three progressively looser criteria: the block wraps the start of a ground-truth block (2.1); it occupies the same local context, measured by overlapping lines before and after both blocks (2.2); or it carries the same edits in the same order at a different location (2.3). Each Pass 2 pairing is recorded with a tester program that applies the predicted block in place of the matched ground-truth blocks, together with the ground-truth fixes for all other bugs. Because one predicted block may cover several ground-truth blocks, the output is a partial many-to-one matching; unmatched predicted blocks count against precision and unmatched ground-truth blocks count against recall.

The procedure $\mathrm{essential}_{\mathcal{U}}$ first runs each tester, marking a pairing as semantically correct for its bugs if the tester passes $F_{\mathcal{U}}$ (exact matches need no test). Each correct pairing on ground-truth blocks $\mathcal{G}$ may then be credited with at most $\kappa(\mathcal{G}) = \sum_{B^{\mathrm{gt}} \in \mathcal{G}} (|B^{\mathrm{gt}}.\Delta| + \epsilon)$ lines, i.e., $|E_i| + \epsilon$ per matched bug i, which reduces to $\epsilon + 1$ for single-line bugs. Finally, a deep redundancy check enumerates contiguous sub-blocks of the predicted block up to this cap, applies each to the buggy program with the ground-truth fixes for the other bugs, and keeps the smallest one that still passes the tests as the essential edit size. Extra predicted lines are thus credited only when the model's fix actually needs them, while wholesale regeneration receives no credit beyond the cap; the qualitative consequences are examined in §5.5.

Algorithm 1: map: mapping predicted edits to ground-truth edits

Input: Buggy program C_b; Predicted edits Δ_{pred}; GT edits Δ_{gt}
Output: Matched blocks $\mathcal{M}$

$\mathcal{M} \leftarrow \emptyset$, $\Delta_{\text{gt}}^{\text{rem}} \leftarrow \Delta_{\text{gt}}$, $\Delta_{\text{pred}}^{\text{rem}} \leftarrow \Delta_{\text{pred}}$ `// each element in Δ has fields: line, edit`
$\mathcal{B}_{\text{gt}}^{\text{all}} \leftarrow$ ParseToBlocks$(\Delta_{\text{gt}}^{\text{rem}})$
$\mathcal{B}_{\text{gt}}^{\text{rem}} \leftarrow$ ParseToBlocks$(\Delta_{\text{gt}}^{\text{rem}})$ `// each block in B has fields: start, end and Δ`

Pass 1: Exact block-level matches (EM).
`// Whole blocks only: a partial fix of a multi-line GT block is not an EM.`
foreach *predicted block* $B^{pred} \in$ ParseToBlocks(Δ_{pred}^{rem}) ***in descending*** *start* **do**
  if $\exists B^{gt} \in \mathcal{B}_{gt}^{rem}$ ***with*** $(B^{gt}.start, B^{gt}.end) = (B^{pred}.start, B^{pred}.end)$ ***and***
   Equal$(B^{pred}.\Delta, B^{gt}.\Delta)$ **then**
    $\mathcal{M} \leftarrow \mathcal{M} \cup \{$MakeMatch$(B^{\text{pred}}, \{B^{\text{gt}}\}, \text{none})\}$ `// No need to test`
    remove the lines of B^{pred} from $\Delta_{\text{gt}}^{\text{rem}}$ and from $\Delta_{\text{pred}}^{\text{rem}}$
    remove B^{gt} from $\mathcal{B}_{\text{gt}}^{\text{rem}}$
$\mathcal{B}_{\text{pred}}^{\text{rem}} \leftarrow$ ParseToBlocks$(\Delta_{\text{pred}}^{\text{rem}})$

Pass 2: Block-level matching.
for $j \leftarrow 1$ **to** $|\mathcal{B}_{pred}^{rem}|$ **do**
  $B^{\text{pred}} \leftarrow \mathcal{B}_{\text{pred}}^{\text{rem}}[j]$
  $\mathcal{G} \leftarrow \emptyset$ `// matched GT blocks for this predicted block`
  (2.1) Wrap match: predicted block covers GT block start.
  foreach $B^{gt} \in \mathcal{B}_{gt}^{rem}$ **do**
    if $B^{pred}.start \leq B^{gt}.start \leq B^{pred}.end$ **then**
      $\mathcal{G} \leftarrow \mathcal{G} \cup \{B^{\text{gt}}\}$
  (2.2) Near match: context-line overlap before and after.
  if $\mathcal{G} = \emptyset$ **then**
    $S_{\text{pred}}^{-} \leftarrow$ ContextBefore$(C_b, B^{\text{pred}}, \mathcal{B}_{\text{pred}}^{\text{rem}})$
    $S_{\text{pred}}^{+} \leftarrow$ ContextAfter$(C_b, B^{\text{pred}}, \mathcal{B}_{\text{pred}}^{\text{rem}})$
    foreach $B^{gt} \in \mathcal{B}_{gt}^{rem}$ **do**
      $S_{\text{gt}}^{-} \leftarrow$ StrideContextBefore(C_b, B^{gt})
      $S_{\text{gt}}^{+} \leftarrow$ StrideContextAfter(C_b, B^{gt})
      if LineSetMatch$(S_{pred}^{-}, S_{gt}^{-})$ ***and*** LineSetMatch$(S_{pred}^{+}, S_{gt}^{+})$ **then**
        $\mathcal{G} \leftarrow \{B^{\text{gt}}\}$
        break
  (2.3) Distant-but-identical: edit-wise equality.
  if $\mathcal{G} = \emptyset$ **then**
    foreach $B^{gt} \in \mathcal{B}_{gt}^{rem}$ **do**
      if $|B^{pred}.\Delta| = |B^{gt}.\Delta|$ ***and*** Equal$(B^{pred}.\Delta.edit, B^{gt}.\Delta.edit)$ **then**
        $\mathcal{G} \leftarrow \{B^{\text{gt}}\}$ `// same edits in the same order, at any location`
        break
  if $\mathcal{G} \neq \emptyset$ **then**
    $B^{\text{test}} \leftarrow$ MergeBlocks$(\mathcal{B}_{\text{gt}}^{\text{all}} \setminus \mathcal{G} \cup \{B^{\text{pred}}\})$
    $C^{\text{test}} \leftarrow$ Apply$(B^{\text{test}}.\Delta, C_b)$
    $\mathcal{M} \leftarrow \mathcal{M} \cup \{$MakeMatch$(B^{\text{pred}}, \mathcal{G}, C^{\text{test}})\}$ `// Use` $C^{\texttt{test}}$ `to test matched pairs`
    remove all blocks in $\mathcal{G}$ from $\mathcal{B}_{\text{gt}}^{\text{rem}}$
return $\mathcal{M}$

Algorithm 2: $\mathrm{essential}_{\mathcal{U}}$: finding ϵ-relaxed essential edits for each matching in $\mathcal{M}$

Input: Buggy program C_b; GT blocks $\mathcal{B}_{\text{gt}}$; Predicted and GT edits $\Delta_{\text{pred}}, \Delta_{\text{gt}}$; Tolerance ϵ; Unit tests $F_{\mathcal{U}}(\cdot)$

Output: Final matching $\mathcal{M}_\epsilon$ with two more additional fields `success` and `essential_size`

Step 1: Candidate matching via map**.**

$\mathcal{M} \leftarrow \mathrm{map}(C_b, \Delta_{\text{pred}}, \Delta_{\text{gt}})$

// $\mathcal{M}$ contains a pred_block B_{pred}, gt_blocks $\mathcal{G}$, and a tester C^{test} built by replacing matched GT blocks with predicted blocks.

$\kappa(\mathcal{G}) \leftarrow \sum_{B^{\text{gt}} \in \mathcal{G}} \left(|B^{\text{gt}}.\Delta| + \epsilon \right)$

// Allowed predicted lines for a match on GT blocks $\mathcal{G}$: $|E_i| + \epsilon$ per matched bug i, i.e., $|\mathcal{G}| \cdot (\epsilon + 1)$ for single-line bugs

Step 2: Semantic equivalence verification using $F_{\mathcal{U}}$**.**

foreach *match record* $r \in \mathcal{M}$ **do**
- **if** r.tester $=$ *none* ***or*** $F_{\mathcal{U}}(r.\texttt{tester}) = 1$ **then**
  - r.success $\leftarrow$ True
  - $B^{\text{pred}} \leftarrow r$.pred_block
  - $\mathcal{G} \leftarrow r$.gt_blocks
  - r.essential_size $\leftarrow \min\left(\kappa(\mathcal{G}), |B^{\text{pred}}.\Delta|\right)$
- **else**
  - r.success $\leftarrow$ False
  - r.essential_size $\leftarrow 0$

Step 3: Deep redundancy check to realize ϵ**-relaxed essential edits.**

foreach *match record* $r \in \mathcal{M}$ ***with*** r.success $=$ True **do**
- $\mathcal{S} \leftarrow \emptyset$ // candidate sub-blocks
- $B^{\text{pred}} \leftarrow r$.pred_block
- $\mathcal{G} \leftarrow r$.gt_blocks
- // Enumerate smaller contiguous sub-edits within the predicted block.
- let $(B^{\text{pred}}.\Delta) = [(\ell_1, v_1), \ldots, (\ell_m, v_m)]$ ordered by ℓ
- **for** $\tau \leftarrow 0$ **to** $\kappa(\mathcal{G}) - 1$ **do**
  - **for** $line \leftarrow 1$ **to** $m - \tau$ **do**
    - $B^{\text{sub}} \leftarrow \textsc{SubBlock}(B^{\text{pred}}, line, line + \tau)$
    - $B^{\text{test}} \leftarrow \textsc{MergeBlocks}(\mathcal{B}_{\text{gt}} \setminus \mathcal{G} \cup \{B^{\text{sub}}\})$
    - $C^{\text{sub}} \leftarrow \textsc{Apply}(B^{\text{test}}.\Delta, C_b)$
    - $\mathcal{S} \leftarrow \mathcal{S} \cup \{(\tau + 1, C^{\text{sub}})\}$
- // Find the smallest τ that still passes $F_{\mathcal{U}}$.
- $\tau^\star \leftarrow +\infty$
- **foreach** $(\tau, C^{sub}) \in \mathcal{S}$ **do**
  - **if** $F_{\mathcal{U}}(C^{sub}) = 1$ ***and*** $\tau < \tau^\star$ **then**
    - $\tau^\star \leftarrow \tau$
- **if** $\tau^\star < +\infty$ **then**
  - r.essential_size $\leftarrow \tau^\star$

$\mathcal{M}_\epsilon \leftarrow \mathcal{M}$

return $\mathcal{M}_\epsilon$

D Examples of Debugging Categories

We supply the qualitative examples behind the failure-mode taxonomy reported in §5.5. We separate examples into two regimes that the unit-test signal alone cannot distinguish: (i) patches that pass the unit tests but receive imperfect precision or recall (Figures 11–17), and (ii) patches that fail the unit tests despite containing partially correct edits (Figures 18–19). Percentages cited in each caption are computed over the 240 manually inspected patches drawn for the metric-verification study.

D.1 Failure modes when unit tests pass

The seven figures below stratify cases where the unit tests pass but the patch is not maximally targeted. Figures 11–14 cover precision-loss patterns (recall $= 1$, precision < 1); Figures 15–17 cover recall-loss patterns (recall < 1, where the recovered fix is functionally equivalent or affected by injection coupling).

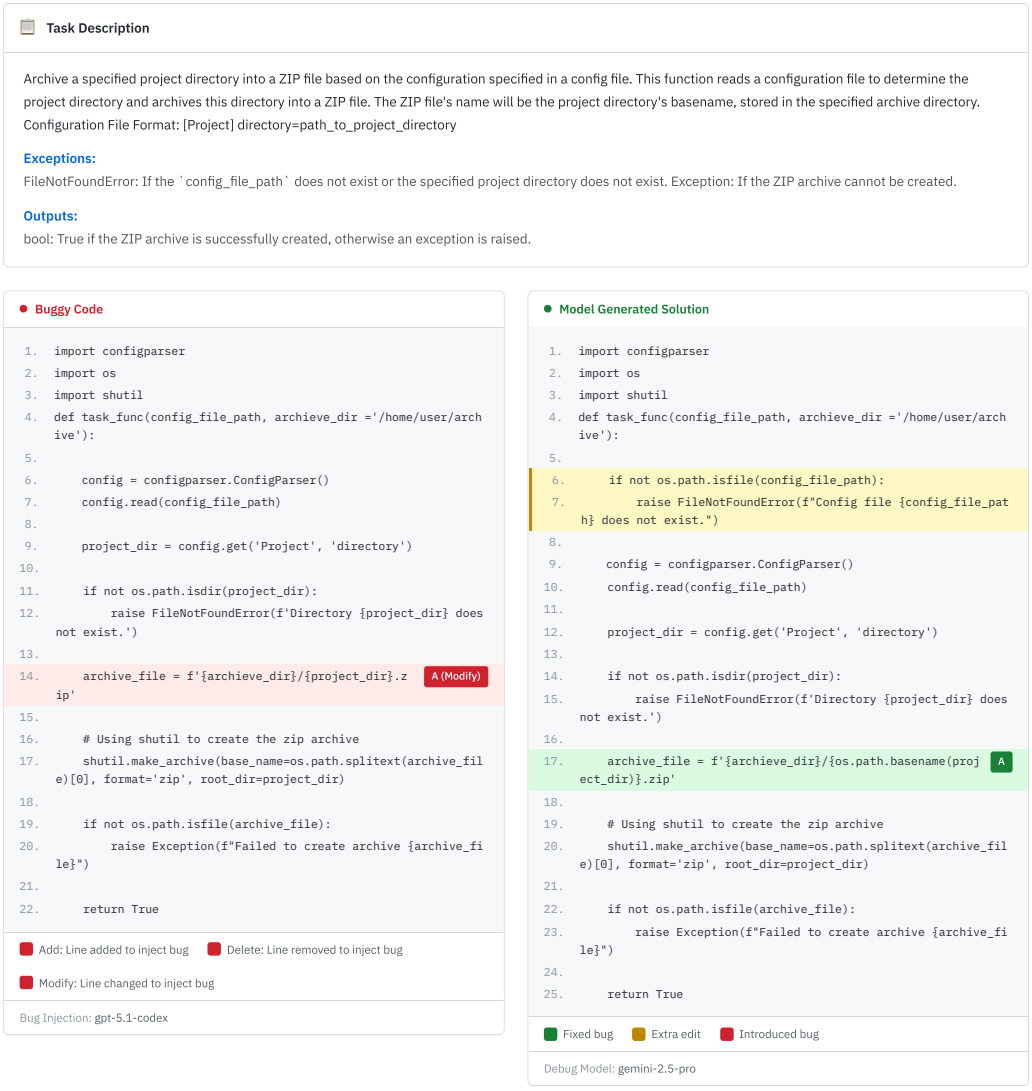


Figure 11: Redundant guard checks (9.8% of recall=1, precision<1 cases). The model adds a defensive check (e.g., a null guard) on top of an already-correct fix; the extra check is irrelevant to the unit tests but counts as a spurious edit.

Figure 11 illustrates the most benign precision loss: the model produces the correct minimal fix and then layers an additional guard on top. The guard is harmless, often even reasonable defensive practice, but it is a strict superset of the ground-truth edit and therefore reduces precision without affecting unit-test outcome.

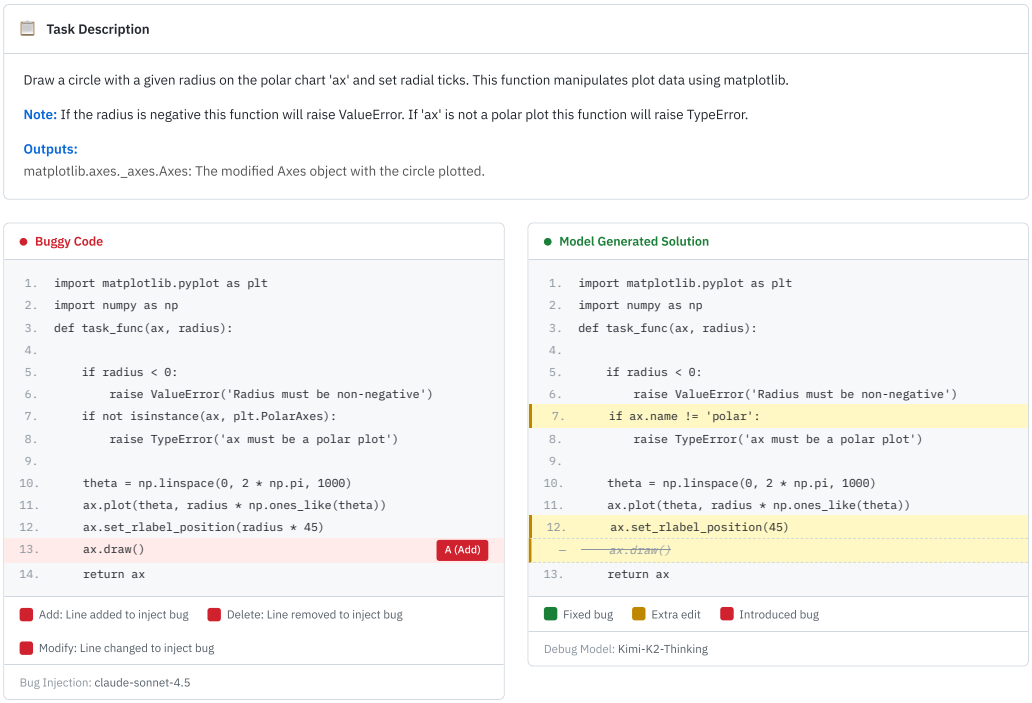


Figure 12: Additional modifications to correct code (66.8% of recall=1, precision<1 cases). The model edits unrelated, already-correct lines, typically renaming variables, reordering statements, or adjusting unrelated control flow, in addition to applying the necessary fix.

Figure 12 captures the dominant precision-loss mode: while the necessary fix is present, the patch also rewrites surrounding correct code. This is the empirical signature of the regeneration-style behaviour discussed in §5; the model treats the buggy program as a draft to revise globally rather than a working program to amend locally.

Figure 13 isolates the most extreme precision failure: the patch is a from-scratch reimplementation rather than an edit. Such patches almost always pass the unit tests by construction (the rewrite is correct) but score precision near zero because no predicted edit aligns with a ground-truth block; the model has effectively bypassed the debugging task.

Figure 14 highlights a category in which low precision is genuinely informative: the model patches an additional, real bug in the seed solution itself. These cases account for under 2% of the inspected patches but illustrate that perfect precision is not always desirable when the ground truth is incomplete; we interpret them as a soft upper bound on how aggressively precision can be optimized without losing useful repairs.

Figure 15 dominates the recall-loss regime. The fix is correct on the test suite, but the predicted edit is structurally far enough from the ground-truth block that map cannot match them. These cases are the target of the relocation-tolerant criteria in Pass 2 of map (near match and distant-but-identical; Appendix C), which pair moved edits with their ground-truth blocks and validate them with unit tests; edits that are both moved and rewritten remain beyond their reach.

Figure 16 captures the irreducible ambiguity that comes from defining recall against a single reference patch. When two or more equally targeted fixes resolve the same bug, the model is penalized for choosing the wrong one even though no over-editing occurred. The 20% prevalence among recall-loss

cases is consistent with bug-pattern diversity in the ODC categories; addressing it would require multi-reference annotations, which we discuss as a future direction in §5.

Figure 17 is the only recall-loss category attributable to dataset construction rather than model behaviour. Composing multiple bug injections risks turning two locally minimal injections into one coupled defect; the model then resolves the pair with a single edit that the matcher counts as missing one of the original bugs. The 1.65% overall prevalence indicates that the bug-independence assumption used by our generator holds in the large majority of cases, and that the metric remains accurate on $>$97.5% of the evaluation data.

D.2 Failure modes when unit tests fail

The two figures below cover the symmetric regime: patches that fail the unit tests despite containing partially correct edits. The taxonomy here is binary, separating cases where the model under-edits from cases where it edits enough but introduces fresh defects.

Figure 18 shows a category that, despite failing tests, exhibits exactly the targeted-edit behaviour the benchmark rewards: the model identifies and fixes some bugs precisely. Failures here are recall-bound rather than precision-bound, and they tend to grow with bug count k, consistent with the per-bug-count breakdown in Figure 8.

Figure 19 reveals the largest single failure category in this regime. The patch hits every ground-truth fix—recall is 1—yet the unit tests still fail, because the model also rewrote correct code and broke it. This is the same regeneration mechanism that drives the precision losses in Figure 12; the difference is that the side effects landed in code paths the test suite happens to exercise. Together with under-repair, regressive repair shows that the unit-test pass rate alone discards information that precision and recall recover.

Task Description

Organize files in a directory based on the first text that is not enclosed in square brackets. Move the files to subdirectories named after this text. If no matching text is found, the file is not moved.

Outputs:

tuple: str: The directory path with organized files. dict: A dictionary where keys are the created subdirectories and values are lists of files moved to them.

Buggy Code

```
1.  import re
2.  import os
3.  import shutil
4.  from datetime import datetime
5.  def task_func(directory):
6.
7.      DATE_FORMAT = '%Y%m%d%H%M%S'
8.      moved_files = {}
9.      for filename in os.listdir(directory):
10.         with open(os.path.join(directory, filename), 'r') as f
    ile:
11.             content = file.read()
12.             match = re.search('(.*?)\[.*?\]', content)
13.             if match:
14.                 subdirectory = match.group(1).strip()
15.
16.                 if not os.path.exists(os.path.join(directory,
    subdirectory)):
17.                     os.makedirs(os.path.join(directory, subdir
    ectory))
18.
19.                 new_filename = f"{filename.split('.')[0]}_{dat
    etime.now().strftime(DATE_FORMAT)}.{filename.split('.')[1]}"
20.                 shutil.move(os.path.join(directory, filename),
    os.path.join(directory, subdirectory, new_filename))
21.
22.                 if subdirectory not in moved_files:
23.                     moved_files[subdirectory] = []
24.                 moved_files[subdirectory].append(new_filename)
25.
26.     return directory.resolve(), moved_files                    A (Modify)
```

Add: Line added to inject bug Delete: Line removed to inject bug Modify: Line changed to inject bug

Bug Injection: claude-sonnet-4.5

Model Generated Solution

```
1.  import re
2.  import os
3.  import shutil
4.  from datetime import datetime
5.  def task_func(directory):
6.
7.      DATE_FORMAT = '%Y%m%d%H%M%S'
8.      moved_files = {}
9.      for filename in os.listdir(directory):
10.         filepath = os.path.join(directory, filename)
11.         if os.path.isfile(filepath):
12.             with open(filepath, 'r') as file:
13.                 content = file.read()
14.                 match = re.search(r'^(.*?)\[(.*?)\](.*)$', con
    tent, re.DOTALL)
15.                 if not match:
16.                     match = re.search(r'\[(.*?)\](.*)$', conte
    nt, re.DOTALL)
17.                     if match:
18.                         before_brackets = content[:match.start
    ()]
19.                         first_non_bracket_text = before_bracke
    ts.split()[-1] if before_brackets.split() else ""
20.                         subdirectory = first_non_bracket_text
21.                     else:
22.                         continue
23.                 else:
24.                     before_first_bracket = match.group(1)
25.                     parts = before_first_bracket.strip().split
    ()
26.                     subdirectory = parts[-1] if parts else ""
27.
28.                 if not subdirectory:
29.                     continue
30.
31.                 if not os.path.exists(os.path.join(directory,
    subdirectory)):
32.                     os.makedirs(os.path.join(directory, subdir
    ectory))
33.
34.                 name_part, ext_part = os.path.splitext(filenam
    e)
35.                 new_filename = f"{name_part}_{datetime.now().s
    trftime(DATE_FORMAT)}{ext_part}"
36.                 shutil.move(filepath, os.path.join(directory,
    subdirectory, new_filename))
37.
38.                 if subdirectory not in moved_files:
39.                     moved_files[subdirectory] = []
40.                 moved_files[subdirectory].append(new_filename)
41.
42.     return os.path.abspath(directory), moved_files             A
```

Fixed bug Extra edit Introduced bug

Debug Model: Qwen3-Coder-480B-A35B-Instruct-FP8

Figure 13: Complete rewrite (7.8% of recall=1, precision<1 cases). The model regenerates the entire function from scratch; the original buggy lines are absent from the patch, replaced by an alternative implementation.

Task Description

Unzip a list of objects and their 3D coordinates, run PCA to reduce the dimensionality to 2D, and depending on the value of save_plot parameter, either save the plot to the provided path and return the 2D coordinates or return the 2D coordinates and the plot's Axes.

Exceptions:
ValueError: If save_plot is True but plot_path is not provided.

Outputs:
coordinates_2d (numpy.ndarray): The 2D coordinates after applying PCA. ax (matplotlib.axes._axes.Axes, optional): The plot's Axes if save_plot is True.

Buggy Code

```
1.  import numpy as np
2.  from sklearn.decomposition import PCA
3.  import matplotlib.pyplot as plt
4.  def task_func(data, save_plot=False, plot_path=None):
5.
6.      items, x_values, y_values, z_values = zip(*data)
7.      coordinates = np.array(list(zip(x_values, y_values, z_valu
    es)))
8.
9.      pca = PCA(n_components=2)
10.     coordinates_2d = pca.fit_transform(coordinates)
11.
12.     # Initialize a fresh plot
13.     plt.figure()
14.     fig, ax = plt.subplots()
15.     ax.scatter(*zip(*coordinates_2d))
16.
17.     if save_plot:
18.         if plot_path is not None == True:          A (Modify)
19.             plt.savefig(plot_path)
20.             plt.close(fig)
21.             return coordinates_2d, ax
22.         else:
23.             raise ValueError("plot_path is required if save_pl
    ot is True")
24.     else:
25.         return coordinates_2d
```

Add: Line added to inject bug Delete: Line removed to inject bug

Modify: Line changed to inject bug

Bug Injection: claude-sonnet-4.5

Model Generated Solution

```
1.  import numpy as np
2.  from sklearn.decomposition import PCA
3.  import matplotlib.pyplot as plt
4.  def task_func(data, save_plot=False, plot_path=None):
5.
6.      items, x_values, y_values, z_values = zip(*data)
7.      coordinates = np.array(list(zip(x_values, y_values, z_valu
    es)))
8.
9.      pca = PCA(n_components=2)
10.     coordinates_2d = pca.fit_transform(coordinates)
11.
12.     # Initialize a fresh plot
13.     plt.figure()
14.     fig, ax = plt.subplots()
15.     ax.scatter(*zip(*coordinates_2d))
16.
17.     if save_plot:
18.         if plot_path is not None:                  A
19.             plt.savefig(plot_path)
20.             plt.close(fig)
21.             return coordinates_2d, ax
22.         else:
23.             raise ValueError("plot_path is required if save_pl
    ot is True")
24.     else:
25.         plt.close(fig)
26.         return coordinates_2d
```

Fixed bug Extra edit Introduced bug

Debug Model: Qwen3-Coder-480B-A35B-Instruct-FP8

Figure 14: Discovering bugs missed by ground truth (1.9% of recall=1, precision<1 cases). The model fixes the injected bug *and* a separate, latent defect in the seed solution that the original benchmark's tests fail to exercise.

Task Description

Analyzes a given DataFrame containing article titles and content to identify articles with titles that include the words "how" or "what". It calculates the TF-IDF scores for the words in the content of these articles and visualizes these scores in a bar plot.

Note: If the DataFrame does not contain 'Title' and 'Content' columns, the function returns an empty plot. If no articles have titles containing "how" or "what," the function also returns an empty plot. Set the name of the y-axis to 'TF-IDF Score'. Set xticks to display the feature names vertically.

Outputs:

Axes: A matplotlib Axes object displaying a bar plot of the TF-IDF scores.

Buggy Code

```
1.  import re
2.  import matplotlib.pyplot as plt
3.  from sklearn.feature_extraction.text import TfidfVectorizer
4.  import numpy as np
5.  def task_func(df):
6.
7.      pattern = re.compile(r'(how|what)', re.IGNORECASE)
8.
9.      # Check if the DataFrame has the required columns
10.     if not set(['Title', 'Content']).issubset(df.columns):
11.         fig, ax = plt.subplots()
12.         return ax
13.
14.     interesting_articles = df[df['Title'].apply(lambda x: bool
    (pattern.search(x)))]
15.
16.     fig, ax = plt.subplots()
17.
18.     # If there are no interesting articles, return an empty pl
    ot
19.     if interesting_articles is None:                          A (Modify)
20.         return ax
21.
22.     vectorizer = TfidfVectorizer()
23.     X = vectorizer.transform(interesting_articles             B (Modify)
    ['Content'])
24.     tfidf_scores = np.array(X.sum(axis=0))[0]
25.
-       ax.bar(vectorizer.get_feature_names_out(), tfidf          C (Delete)
    _scores)
26.     ax.set_ylabel('TF-IDF Score')
27.     plt.xticks(rotation='vertical')
28.
29.     return ax
```

Add: Line added to inject bug Delete: Line removed to inject bug

Modify: Line changed to inject bug

Bug Injection: gemini-2.5-pro

Model Generated Solution

```
1.  import re
2.  import matplotlib.pyplot as plt
3.  from sklearn.feature_extraction.text import TfidfVectorizer
4.  import numpy as np
5.  def task_func(df):
6.
7.      pattern = re.compile(r'(how|what)', re.IGNORECASE)
8.
-       # Check if the DataFrame has the required columns
9.      if not set(['Title', 'Content']).issubset(df.columns):
10.         fig, ax = plt.subplots()
11.         return ax
12.
13.     interesting_articles = df[df['Title'].apply(lambda x: bool
    (pattern.search(x)))]
14.
15.     fig, ax = plt.subplots()
16.
17.     if interesting_articles.empty:                            A
-       if interesting_articles is None:                          A
18.         return ax
19.
20.     vectorizer = TfidfVectorizer()
21.     X = vectorizer.fit_transform(interesting_articles['Con    B
    tent'])
22.     tfidf_scores = np.array(X.sum(axis=0))[0]
23.     feature_names = vectorizer.get_feature_names_out()
24.
25.     ax.bar(feature_names, tfidf_scores)                       C
26.     ax.set_ylabel('TF-IDF Score')
27.     ax.set_xticks(range(len(feature_names)))
28.     ax.set_xticklabels(feature_names, rotation='vertical')
29.
30.     return ax
```

Fixed bug Extra edit Introduced bug

Debug Model: gpt-5.1-codex

Figure 15: Functionally correct but undetected (70% of recall<1 cases). The model's edit is semantically equivalent to the ground-truth fix but expressed at a different location or in a different syntactic form, so the strict block matcher fails to align it.

Task Description

Perform a linear regression between "age" and "score" in the DataFrame, excluding rows with duplicate names. Plot the regression line and the scatter plot of the data.

Note: The function use "Linear Regression" for the plot title. The function use "Age" and "Score" as the xlabel and ylabel respectively.

Exceptions:
The function will raise a ValueError is input df is not a DataFrame.

Outputs:
tuple: A tuple containing the matplotlib.pyplot object and the axes object.

Buggy Code

```
1.  import pandas as pd
2.  from scipy import stats
3.  import matplotlib.pyplot as plt
4.  def task_func(df):
5.
6.      if not isinstance(df, pd.DataFrame):
7.          raise ValueError("The input df is not a DataFrame")
8.
9.      df = df.drop_duplicates(subset='Name')
10.
11.     slope, intercept, r_value, _, _ = stats.linregress(df['Ag
    e'], df['Score'])
12.
-       df['Age_up'] = intercept + slope * df['Age']                A (Delete)
13.     fig = plt.figure(figsize=(8, 6))
14.     ax = fig.add_subplot(111)
15.     plt.scatter(df['Age'], df['Score'], label='Data')
-       plt.plot(df['Age'].values, df['Age_up'].values,             B (Delete)
    'r', label='Fitted line')
16.     plt.xlabel('Age')
17.     plt.ylabel('Score')
18.     plt.title('Linear Regression')
19.     plt.legend()
20.     return plt, ax
```

Add: Line added to inject bug Delete: Line removed to inject bug

Modify: Line changed to inject bug

Bug Injection: claude-sonnet-4.5

Model Generated Solution

```
1.  import pandas as pd
2.  from scipy import stats
3.  import matplotlib.pyplot as plt
4.  def task_func(df):
5.
6.      if not isinstance(df, pd.DataFrame):
7.          raise ValueError("The input df is not a DataFrame")
8.
9.      df = df.drop_duplicates(subset='Name')
10.
11.     slope, intercept, r_value, _, _ = stats.linregress(df['Ag
    e'], df['Score'])
12.
13.     fig = plt.figure(figsize=(8, 6))
14.     ax = fig.add_subplot(111)
15.     plt.scatter(df['Age'], df['Score'], label='Data')
16.     plt.plot(df['Age'], slope * df['Age'] + intercept, col      B
    or='red', label='Regression Line')
17.     plt.xlabel('Age')
18.     plt.ylabel('Score')
19.     plt.title('Linear Regression')
20.     plt.legend()
21.     return plt, ax
```

Fixed bug Extra edit Introduced bug

Debug Model: grok-code-fast-1

Figure 16: Multiple minimal fixes (20% of recall<1 cases). The injected bug admits more than one minimal correct repair, and the model selects a fix that disagrees with the ground-truth choice while remaining minimal.

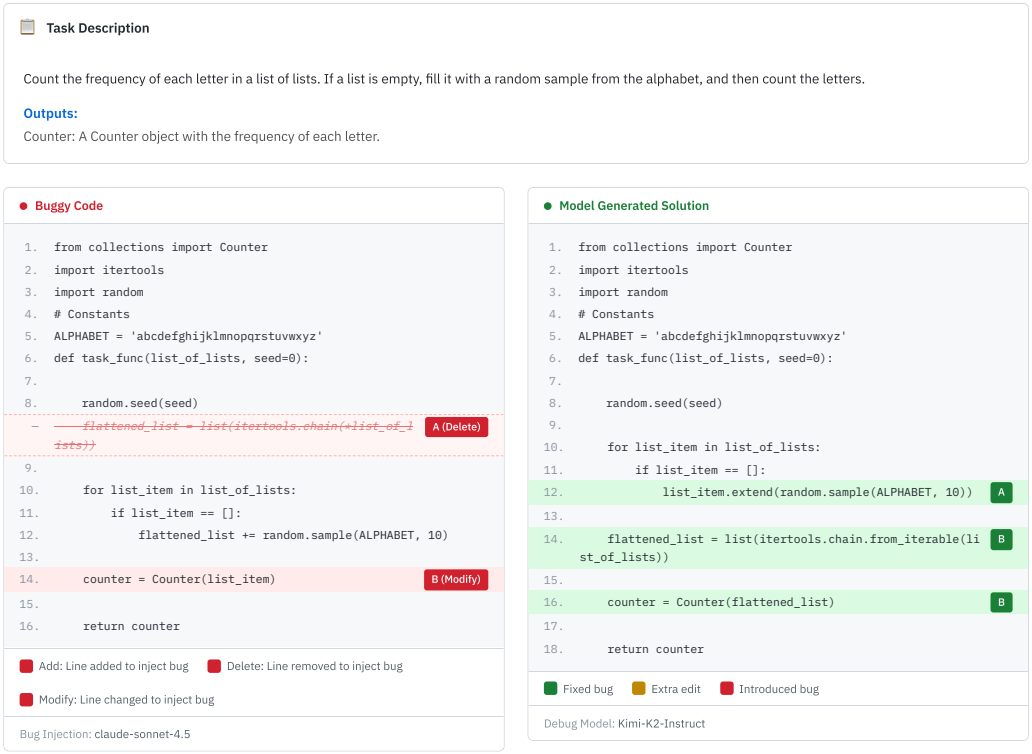


Figure 17: Bug-composition artefacts (10% of recall$<$1 cases, $\approx$1.65% overall). When multiple bugs are injected into the same program, an upstream injected bug occasionally changes the program's logic enough that a downstream injected bug becomes vacuous or fuses with the upstream fix.

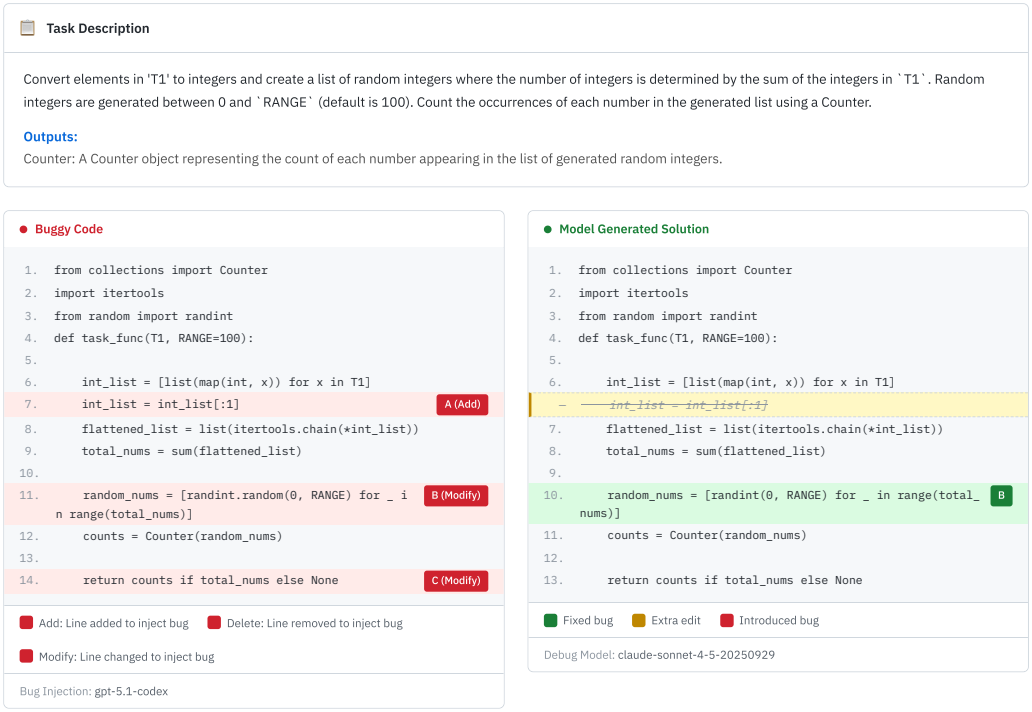


Figure 18: Under-repair (31.4% of failing cases). The model applies a subset of the necessary fixes without introducing extraneous edits (precision $= 1$), but at least one ground-truth bug is left untouched (recall < 1), so the unit tests fail.

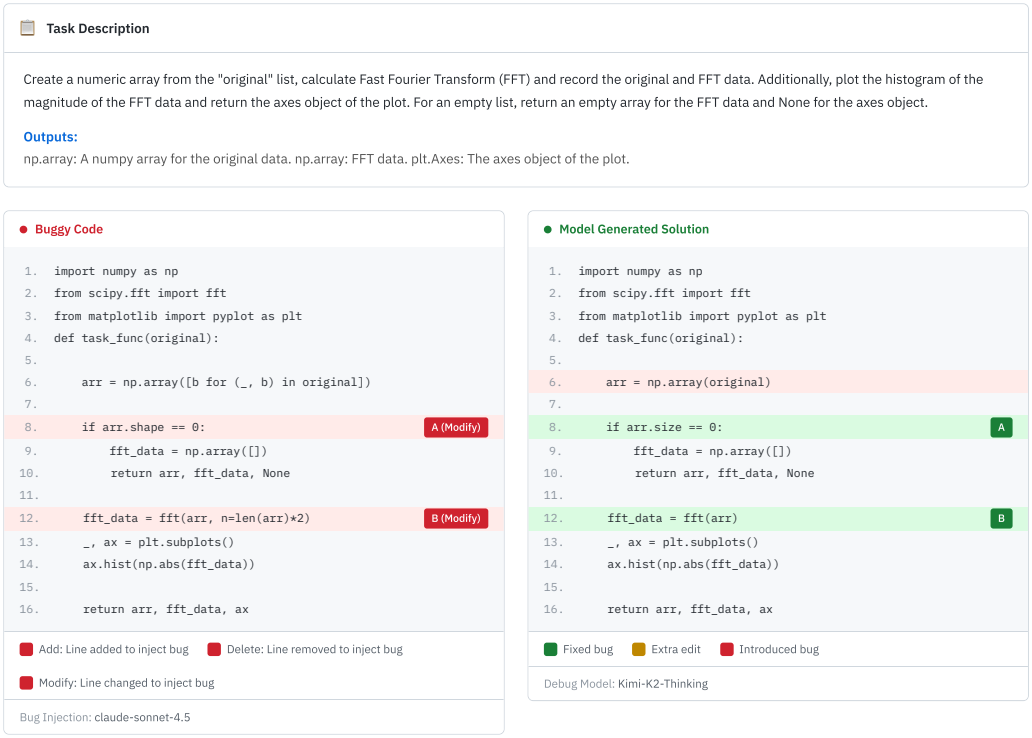


Figure 19: Regressive repair (39.2% of failing cases). The model successfully resolves all originally injected bugs (recall = 1) but introduces new defects elsewhere in the patch that cause the unit tests to fail.

E Prompt templates

We provide the prompt templates used in our experiments, ranging from bug injection and solution rewriting to minimal and free-form debugging with optional unit tests and execution feedback, shown in Figures 20–31.

```
Your task is to perform a deep analysis of a code snippet and intentionally introduce ONE bug.
You will be given two major components:
PART 1: A task description outlining the intended functionality.
PART 2: A solution to the task.
First, carefully read and understand both the task description and the provided solution.
Then, modify the solution by injecting a realistic programming error.
You will be asked to introduce one of the following bug types into the code: Assignment, Checking, Algorithm,
Build/Package/Merge or Timing/Serialization.
You will be asked to perform exactly one action: Add one line, Delete one line or Modify one line.

Instructions for modifying the code:
- ONLY modify ONE selected line to induce a HARD bug to the task.
- Keep the other lines of the code solution EXACTLY the SAME.
- Do NOT add any new comments to the modified line.
- DO NOT introduce easy bugs such as referencing variable names before declaration, adding typos or commas.

You should output two things:
- The subtype of the introduced bug.
- A buggy code snippet with only a ONE line difference with the original code solution, and no comments on that
modified line.

---

task_prompt: {task_prompt}
correct_solution: {correct_solution}
bug_type: {bug_type}
action_on_lines: {action_on_lines}
```

task_prompt
The programming task description for context

correct_solution
A correct Python code solution

bug_type
The type of bug to add

action_on_lines
The action and the lines to choose from

Figure 20: Bug injection prompt for benchmark construction.

```
Debug the given Python code that contains errors. ONLY fix the bugs. Make minimal edits.
Do NOT generate a new solution based on the problem description.
Do NOT reformat lines that are already correct.
Do NOT edit or add any comments.

The input consists of two parts:
- A problem description outlining the intended functionality.
- A buggy code that needs to be fixed.

Your response should include:
- A self-contained, corrected Python implementation, with minimal edits.

---

problem_description: {problem_description}
buggy_solution: {buggy_solution}
```

problem_description
The problem description

buggy_solution
The buggy solution

Figure 21: Minimal debugging prompt with problem description and buggy code.

```
Debug the given Python code that contains errors. ONLY fix the bugs. Make minimal edits.
Do NOT generate a new solution based on the problem description.
Do NOT reformat lines that are already correct.
Do NOT edit or add any comments.

The input consists of three parts:
- A problem description outlining the intended functionality.
- A buggy code that needs to be fixed.
- A set of unit tests for the problem.

Your response should include:
- A self-contained, corrected Python implementation, only making minimal edits on the buggy code.

---

problem_description: {problem_description}
buggy_solution: {buggy_solution}
unit_tests: {unit_tests}
```

problem_description
The problem description

buggy_solution
The buggy solution

unit_tests
The unit tests

Figure 22: Minimal debugging prompt with unit tests.

```
Debug the given Python code that contains errors. ONLY fix the bugs. Make minimal edits.
Do NOT generate a new solution based on the problem description.
Do NOT reformat lines that are already correct.
Do NOT edit or add any comments.

The input consists of three parts:
- A problem description outlining the intended functionality.
- A buggy code that needs to be fixed.
- Previously failed attempts and optionally error feedback.

Your response should include:
- A self-contained, corrected Python implementation, only making minimal edits on the buggy code.

---

problem_description: {problem_description}
buggy_solution: {buggy_solution}
failed_attempts: {failed_attempts}
```

problem_description
The problem description

buggy_solution
The buggy solution

failed_attempts
Previous attempts that failed unit tests, AVOID THEM!

Figure 23: Minimal debugging prompt with execution feedback.

```
Debug the given Python code that contains errors. ONLY fix the bugs. Make minimal edits.
Do NOT generate a new solution based on the problem description.
Do NOT reformat lines that are already correct.
Do NOT edit or add any comments.

The input consists of four parts:
- A problem description outlining the intended functionality.
- A buggy code that needs to be fixed.
- A set of unit tests for the problem.
- Previously failed attempts and optionally error feedback.

Your response should include:
- A self-contained, corrected Python implementation, only making minimal edits on the buggy code.

---

problem_description: {problem_description}
buggy_solution: {buggy_solution}
unit_tests: {unit_tests}
failed_attempts: {failed_attempts}
```

problem_description
The problem description

buggy_solution
The buggy solution

unit_tests
The unit tests

failed_attempts
Previous attempts that failed unit tests, AVOID THEM!

Figure 24: Minimal debugging prompt with unit tests and execution feedback.

```
Debug the given Python code that contains errors. Do NOT add any comments. The input consists of two parts:
- A problem description outlining the intended functionality.
- A buggy code that needs to be fixed.

Your response should include:
- A self-contained, corrected Python implementation.

---

problem_description: {problem_description}
buggy_solution: {buggy_solution}
```

problem_description
The problem description

buggy_solution
The buggy solution

Figure 25: Free-form debugging prompt without minimal edit constraint.

```
Debug the given Python code that contains errors. Do NOT add any comments. The input consists of three parts:
- A problem description outlining the intended functionality.
- A buggy code that needs to be fixed.
- A set of unit tests for the problem.

Your response should include:
- A self-contained, corrected Python implementation.

---

problem_description: {problem_description}
buggy_solution: {buggy_solution}
unit_tests: {unit_tests}
```

problem_description
The problem description

buggy_solution
The buggy solution

unit_tests
The unit tests

Figure 26: Free-form debugging prompt with unit tests.

```
Debug the given Python code that contains errors. Do NOT add any comments. The input consists of three parts:
- A problem description outlining the intended functionality.
- A buggy code that needs to be fixed.
- Previously failed attempts and optionally error feedback.

Your response should include:
- A self-contained, corrected Python implementation.

---

problem_description: {problem_description}
buggy_solution: {buggy_solution}
failed_attempts: {failed_attempts}
```

problem_description
The problem description

buggy_solution
The buggy solution

failed_attempts
Previous attempts that failed unit tests, AVOID THEM!

Figure 27: Free-form debugging prompt with execution feedback.

```
Debug the given Python code that contains errors. Do NOT add any comments. The input consists of four parts:
- A problem description outlining the intended functionality.
- A buggy code that needs to be fixed.
- A set of unit tests for the problem.
- Previously failed attempts and optionally error feedback.

Your response should include:
- A self-contained, corrected Python implementation.

---

problem_description: {problem_description}
buggy_solution: {buggy_solution}
unit_tests: {unit_tests}
failed_attempts: {failed_attempts}
```

problem_description
The problem description

buggy_solution
The buggy solution

unit_tests
The unit tests

failed_attempts
Previous attempts that failed unit tests, AVOID THEM!

Figure 28: Free-form debugging prompt with unit tests and execution feedback.

```
Debug the given Python code that contains errors. ONLY fix the bugs. Make minimal edits.
Do NOT generate a new solution based on the problem description.
Do NOT reformat lines that are already correct.
Do NOT edit or add any comments.

The input consists of three parts:
- A problem description outlining the intended functionality.
- A buggy code that needs to be fixed.
- A set of unit tests for the problem.

Your response should include:
- A self-contained, corrected Python implementation, only making minimal edits on the buggy code.

PART 1: Problem Description
```text
{task_prompt}
```

PART 2: Buggy Code
```python
{buggy_code}
```

PART 3: Unit Tests (context only)
```python
{unit_tests_code}
```

Output format (follow *exactly*):
```python
[Corrected code here]
```
Corrected Code Output (use the format above):
```

task_prompt
Problem description text

buggy_code
Python code with bugs

unit_tests_code
Unit test code

Figure 29: External API template for minimal debugging.

```
Debug the given Python code that contains errors. Do NOT add any comments. The input consists of three parts:
- A problem description outlining the intended functionality.
- A buggy code that needs to be fixed.
- A set of unit tests for the problem.

Your response should include:
- A self-contained, corrected Python implementation.

PART 1: Problem Description
```text
{task_prompt}
```

PART 2: Buggy Code
```python
{buggy_code}
```

PART 3: Unit Tests (context only)
```python
{unit_tests_code}
```

Output format (follow *exactly*):
```python
[Corrected code here]
```
Corrected Code Output (use the format above):
```

task_prompt
Problem description text

buggy_code
Python code with bugs

unit_tests_code
Unit test code

Figure 30: External API template for free-form debugging.

```
Your task is to rewrite the solution code of a task with structural and stylistic perturbations.
You will be given two parts:
PART 1: A task description outlining the intended functionality.
PART 2: A correct solution to the task.

First, read both the task description and the provided solution to understand what the code is supposed to do.
Then, your task is to perform a deep rewriting (perturbation) of the solution code WITHOUT changing its
functionality.
Strictly adhere to the following rules:
    - Do NOT change the starter code as given in the task description.
    - Do NOT add any new comments.
    - Do NOT make shallow redundancy edits, such as a = 1 + 2 - 1.
    - Do NOT condense the code to very short format.
    - NEVER rename variables to very short names (e.g., sum_production -> p), but you can give variables wrong
names.
    - The rewritten code should resemble what human will write and should NOT be hard for human to read.
    - The rewritten code should be different enough from the original code.
You can probably use the following hints:
    - Change loop syntax (e.g., for to while or vice versa, if applicable)
    - Convert recursion to iteration or vice versa (if functionally equivalent)
    - Invert control flow where logical structure remains the same (e.g., replace `if not ...` with an inverted
block)
    - Merge or flatten adjacent or nested if blocks.

Your response should ONLY contain the rewritten Python code, which is a different but correct solution to the
task.

---

task_description: {task_description}
original_solution: {original_solution}
```

task_description
The programming task description.

original_solution
The original Python code solution.

Figure 31: Solution rewriting prompt for benchmark construction.

F Checklist Information

Risks of malicious use of PDB pipeline. PDB provides a systematic procedure for producing realistic buggy programs from existing code by prompting deliberate fault introduction. Hence, the same pipeline that supports controlled debugging evaluation could be repurposed for malicious bug-injection at scale, enabling automated generation of large quantities of plausible faulty code with minimal surface changes. Such capability may be misused to degrade software reliability in collaborative development settings, increase the review burden on maintainers, or seed low-quality code into shared repositories.

Another risk concern is potential data poisoning and model capability shaping. Because PDB converts coding data into structured buggy program and solution pairs, it can lower the cost of creating large synthetic corpora that contain intentional buggy programs with minimal-edit transformations. If used outside the intended research context, these data could be silently employed to bias training toward behaviors that facilitate code degradation, or to contaminate downstream datasets used for model deployment and benchmarking. Even when the immediate artifacts are non-sensitive, the potential, silent shift in model behavior raises concerns.

Licensing landscape of evaluated models. The governance of the evaluated LLMs and benchmarks reveals a sharp dichotomy between proprietary services and open-weight ecosystems. The proprietary tier includes GPT-5.1-Codex (OpenAI), Claude-Sonnet-4.5 (Anthropic), Gemini-2.5-Pro (Google DeepMind), and Grok-Code-Fast (xAI), all of which are accessible exclusively via commercial APIs. These systems are governed by restrictive Terms of Service that prohibit model weight extraction, reverse engineering, and competitive distillation, serving to protect their respective architectural innovations and agentic harnesses. In contrast, the open-weight landscape is characterized by permissive licensing designed to commoditize reasoning capabilities: DeepSeek-V3.2 and its reasoning variant DeepSeek-V3.2-Thinking are released under the MIT License, while Qwen3-Coder-480B utilizes the Apache License 2.0, which includes an explicit patent grant. A hybrid governance model is observed in Kimi-K2-Thinking and Kimi-K2-Instruct, which operate under a Modified MIT License; this variant permits general commercial use but mandates strictly visible attribution for entities exceeding 100 million monthly active users or $20 million in monthly revenue.

Licensing landscape of datasets. Regarding evaluation frameworks, BigCodeBench is governed by the Apache License 2.0, whereas LiveCodeBench adopts a split licensing model with its codebase under the MIT License and its dataset artifacts available under the Creative Commons Attribution 4.0 International License (CC-BY 4.0).

G Limitation

First, our current prompts and data generation procedures target Python programs. While this choice reflects the prevalence of Python in existing coding benchmarks, it may limit immediate applicability to other programming languages. That said, the underlying Orthogonal Defect Classification (ODC) categories are language-agnostic, and adapting PDB to new languages primarily requires modifying in-context examples and language-specific sub-categories, rather than redesigning the framework.

Second, this work focuses on benchmark construction and evaluation, but does not train or post-train models using PDB signals. Future work should incorporate PDB into supervised fine-tuning, reinforcement learning, or self-improvement pipelines to test whether precision-aware objectives can reduce over-editing while preserving functional correctness.

Finally, although our edit-level precision and bug-level recall metrics provide a more accurate characterization of debugging behavior than unit-test–only evaluation, they may still fail to capture certain correct but semantically equivalent fixes. Incorporating more flexible semantic evaluation mechanisms, such as LLM-as-a-judge, may help address these edge cases. More broadly, reliably evaluating semantic correctness of code edits remains an open problem.